TU Clausthal

Master's Thesis

by

Yu Zhang

Explain Reality: Grounded Procedural Tutoring with Foundation Models and Visual Fact Extraction in Immersive Simulation

1st Supervisor: **Prof. Dr. Christian Bartelt**

2nd Supervisor: **Prof. Dr. Rüdiger Ehlers**

5th June 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

I have read and understood the guidelines of the Technical University of Clausthal and those published in the ISSE bulletin, including those regarding the use of literature and other sources. I confirm that I have prepared this thesis independently by myself. Any information taken from other sources and being reproduced in this thesis is clearly referenced.

In terms of the general examination regulations, this work has not yet been submitted to any other examination division.

I hereby agree that my master's thesis may be exhibited in the institute's library and kept for inspection.

Clausthal-Zellerfeld, 5th June 2026	
Location, Date	Yu Zhang

Abstract

Procedural learning in high-fidelity simulation requires learners to execute ordered, state-dependent sequences while verifying intermediate system states, yet existing training aids either break immersion or lack access to live simulator state. This thesis addresses the gap by designing a telemetry-grounded tutoring runtime that integrates procedure-pack-driven state tracking, retrieval-augmented help generation, and a fine-tuned vision-language model (VLM) for structured visual fact extraction from cockpit displays. A central finding is that visual fact ontology design governs model reliability: evolving from an 8-fact to a 13-fact ontology by decomposing compound targets into locally verifiable visual atoms reduced critical false positives by 64–89% on independent holdout sets. A Qwen3.5-9B LoRA adapter trained on 928 bilingual rows achieves 0.99 fact accuracy on two holdout sets with verified zero training overlap; the identical data recipe transfers to Gemma-4-31B with consistent gains, suggesting the approach is not tied to a single backbone. A small formative VR pilot study with nine participants provides descriptive evidence: all four tutor-condition participants completed the full 33-step procedure, whereas none of the five without-tutor participants did; omission errors were largely replaced by assistance-coupled errors in the tutor condition. Controlled evaluation with larger samples remains future work.

Zusammenfassung

Prozedurales Lernen in hochrealistischer Simulation erfordert das Ausführen geordneter, zustandsabhängiger Handlungsabfolgen bei gleichzeitiger Überprüfung von Systemzuständen. Herkömmliche Trainingshilfen unterbrechen jedoch entweder die Immersion oder haben keinen Zugriff auf den Simulatorzustand. Diese Arbeit begegnet dieser Lücke durch eine telemetriegestützte Tutoring-Laufzeitumgebung, die paketbasierte Zustandsverfolgung, Retrieval-Augmented-LLM-Hilfegenerierung und ein feinabgestimmtes Vision-Language-Model (VLM) zur strukturierten Extraktion visueller Fakten aus Cockpit-Anzeigen kombiniert. Ein zentrales Ergebnis ist, dass das Design der visuellen Fakten-Ontologie die Modellzuverlässigkeit bestimmt: Die Weiterentwicklung von einer 8-Fact- zu einer 13-Fact-Ontologie durch Zerlegung zusammengesetzter Ziele in lokal überprüfbare visuelle Atome reduzierte kritische False-Positives um 64–89% auf unabhängigen Holdout-Sets. Ein Qwen3.5-9B LoRA-Adapter, trainiert mit 928 bilingualen Datenzeilen, erreicht eine Fact-Genauigkeit von 0,99 auf zwei Holdout-Sets mit verifizierter Null-Überlappung zum Training; dieselbe Datenrezeptur überträgt sich mit konsistenten Verbesserungen auf Gemma-4-31B, was darauf hindeutet, dass der Ansatz nicht an ein einzelnes Backbone gebunden ist. Eine kleine formative VR-Pilotstudie mit neun Teilnehmern liefert deskriptive Evidenz: Alle vier Teilnehmer mit Tutor absolvierten die vollständige 33-Schritt-Prozedur, während keiner der fünf Teilnehmer ohne Tutor dies schaffte; Auslassungsfehler wurden in der Tutor-Bedingung weitgehend durch assistenzgekoppelte Fehler ersetzt. Eine kontrollierte Evaluation mit größeren Stichproben bleibt zukünftige Arbeit.

Acknowledgements

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation and Problem Statement

Procedural learning—the acquisition of fixed, ordered sequences of actions that must be executed correctly and in the right context—presents challenges that distinguish it from declarative or conceptual learning. A learner must not only memorise the steps but also recognise when each step applies, verify that its preconditions are satisfied, confirm completion before advancing, and recover from errors without propagating mistakes through the remainder of the procedure. In high-fidelity simulation environments, these challenges are compounded by cockpit complexity, instrument density, and the cognitive load of maintaining situational awareness while executing an unfamiliar sequence.

Traditional training aids—video walkthroughs, printed checklists, instructor-led demonstrations—are effective for initial familiarisation but break the continuity of in-simulator practice. A learner who pauses to consult a manual or replay a video segment must context-switch away from the cockpit, losing immersion and, in some cases, the transient simulator state that motivated the consultation. Conversely, ungrounded text-based assistance from a large language model (LLM) risks providing fluent but factually incorrect guidance, because the model has no access to the current simulator state.

Between these extremes lies the need for *grounded, context-aware* tutoring: a system that observes the learner's actions through live simulator telemetry, retrieves relevant documentation, extracts visual evidence from the cockpit where telemetry is insufficient, and delivers verifiable guidance without disrupting the training flow.

The F/A-18C cold-start procedure in DCS World serves as the training scenario for this thesis. The core procedure studied in the benchmark and evaluation work comprises 25 steps (S01–S25) grouped into six operational phases, from battery power-up through engine start, avionics configuration, flight-control checks, and taxi preparation (described in §2.1). The current procedure pack has been extended to S01–S33 to cover additional pre-taxi checks; the extension lies outside the scope of the benchmark and VLM evaluation reported here. Each step carries explicit preconditions that depend

on the completion of earlier steps and on specific telemetry conditions. These inter-step dependencies make the cold-start a representative instance of sequence-sensitive, state-dependent procedural tasks that characterise simulator-based training in aviation, maritime operations, industrial maintenance, and medical procedure instruction.

The central premise of this work is that a tutoring system can be made more reliable and auditable by grounding every actionable output in explicit evidence: a telemetry variable, a gating-rule evaluation, a retrieved document snippet, or a visual fact observation. This constraint serves as both an architectural commitment and a safety requirement: for cockpit procedures, an incorrectly highlighted control or a misidentified procedure state could confuse or mislead the learner.

1.2 Research Questions

The thesis is guided by one main research question and three subordinate questions:

Main RQ. How can an evidence-grounded multimodal tutoring system support state-dependent procedural training in an immersive flight simulator?

RQ1 — System architecture and evidence integration. How can simulator telemetry, retrieved procedural knowledge, and visual fact observations be integrated so that tutoring outputs are state-aware, auditable, and safe for procedural guidance?

RQ2 — VLM adaptation and ontology design. To what extent can a small-data LoRA-adapted VLM reliably extract structured cockpit visual facts, and how does visual fact ontology design affect critical false positives?

RQ3 — Formative VR pilot evidence. In a VR-based F/A-18C cold-start training task, what formative evidence does a small pilot study provide about task completion, procedural errors, and tutor interaction in a with-tutor versus without-tutor comparison?

RQ1 and RQ2 are addressed through the design, implementation, and technical evaluation presented in this thesis. RQ3 is addressed through a completed formative VR pilot study ($n = 9$) and a documented evaluation framework; larger controlled human-subject evaluation remains future work.

1.3 Thesis Vision and Scope

The research vision for this thesis, as articulated in the research proposal [7], centred on a VR-first grounded tutoring system operating on the Meta Quest 3 with DCS-VR. The planned system would receive learner interactions through gaze, hand tracking, and voice input within the VR cockpit; retrieve relevant manual and checklist excerpts; generate concise, citation-backed step guidance through a retrieval-augmented LLM; and render the guidance as in-headset step cards or overlay annotations. The proposal specified a three-condition human-subject evaluation comparing a video-and-notes baseline, an ungrounded text-only LLM, and the grounded tutoring system, with metrics including completion rate, procedural error rate, task time, NASA-TLX workload scores, pre/post knowledge quizzes, and retention and transfer tests.

The current implementation delivers the technical foundations of this vision but differs from the proposal in three important respects. First, the primary runtime has been developed on desktop DCS rather than VR-native Quest 3, because the telemetry, VLM, and overlay infrastructure required a stable development and debugging environment. Second, substantial engineering effort has been invested in components that the proposal did not anticipate: a visual fact extraction pipeline with LoRA fine-tuning, an evidence-grounding architecture enforced at the JSON Schema level, and a systematic benchmark protocol with verified holdout independence. Third, the VR code path has been implemented and verified functional (VR_MIRROR, tutor text display, composite-panel capture). A formative pilot study has been conducted in VR (Section 6.3); larger controlled evaluation remains future work.

The scope of this thesis is therefore threefold:

1. The design and implementation of a telemetry-grounded tutoring runtime with integrated VLM visual fact extraction (Chapters 3, 4, 5);

2. The technical validation of the VLM adaptation pipeline through systematic benchmark evaluation on independent holdout sets (Chapter 6, Part A);

3. A formative VR pilot study ($n = 9$) providing descriptive evidence on task completion, procedural errors, and tutor interaction under with-tutor versus without-tutor conditions, supported by experiment instrumentation and a documented evaluation framework (Chapter 6, Part B).

1.4 Current Implementation Scope

The current implementation is a dual-platform tutoring runtime comprising four integrated subsystems.

Telemetry grounding. The telemetry subsystem ingests raw DCS-BIOS UDP frames and processes them through variable resolution, delta sanitisation, and delta aggregation to produce a stable, normalised state representation. Source-missing tracking distinguishes "the value is zero" from "the value is unknown," a distinction that is critical for correct gating rule evaluation.

VLM visual fact extraction. A visual fact extraction pipeline addresses the gap left by DCS-BIOS telemetry: several procedure steps require visual confirmation of display-page content that is not exported by the simulator. The pipeline comprises structured ontology design, dataset curation with human review, bilingual LoRA fine-tuning, and automated benchmark evaluation across two model backbones (Qwen3.5-9B and Gemma-4-31B).

Evidence-grounded help generation. Every overlay target and step recommendation carries a traceable evidence reference—a telemetry variable, a gating rule evaluation, a retrieved knowledge snippet, or a visual fact observation—enforced at the schema level through runtime-generated JSON Schema rules. A deterministic fallback path ensures the system remains functional when no model is available.

Procedure pack and infrastructure. Procedure definitions, gating rules, telemetry mappings, error taxonomies, and vision fact ontologies are organised as declarative YAML configuration (a *procedure pack*). The system includes event logging and replay infrastructure, an automated scoring engine, experiment export tooling, and a Windows experiment launcher for study deployment.

1.5 Contributions

This thesis makes three contributions:

1. **An evidence-grounded procedural tutoring architecture.** The architecture integrates simulator telemetry, retrieved procedural knowledge, and struc-

tured visual facts within a single help cycle. Every overlay target and step recommendation must be traceable to a specific evidence source, enforced through runtime-generated JSON Schema validation rather than post-hoc logging. The architecture follows a ports-and-adapters pattern with a deterministic fallback path, and all procedure logic is expressed as declarative pack configuration rather than encoded in prompts or code. This contribution is described in Chapters 3 and 5, and evaluated in Chapter 6, Section 6.2.

2. **A visual fact ontology and small-data VLM adaptation pipeline.** The thesis presents a 13-fact visual ontology for F/A-18C cockpit state extraction, evolved from an 8-fact v1 baseline through systematic decomposition of compound procedural targets into locally verifiable visual atoms. The adaptation pipeline spans DCS screenshot capture, AI pre-labeling, human review in Label Studio, bilingual supervised fine-tuning, LoRA fine-tuning, and automated base-vs-LoRA benchmarking on independent holdout sets with verified zero training overlap. A Qwen3.5-9B LoRA adapter trained on 928 bilingual rows achieves 0.99 fact accuracy on two independent holdouts and reduces critical false positives by 64–89%. The identical data recipe transfers to Gemma-4-31B with consistent gains, suggesting the approach is not tied to a single backbone. The ontology evolution yields a transferable design principle: VLMs should extract observable visual evidence, while procedural interpretation belongs in downstream reasoning. This contribution is described in Chapters 3, 4, and evaluated in Chapter 6, Sections 6.1.2 and 6.1.3.

3. **A VR procedural training evaluation framework and formative pilot study.** The thesis delivers study-ready instrumentation for evaluating grounded procedural tutoring in immersive simulation: experiment export tooling, a Windows launcher with session management, baseline and passive condition configurations, semi-automated pre-scoring, error coding guides, and an experiment operation manual. A formative VR pilot study ($n = 9$) was conducted with four with-tutor and five without-tutor participants. All four tutor-condition participants completed the full 33-step procedure; none of the five without-tutor participants did. With-tutor errors were predominantly assistance-coupled (CO/PA), while without-tutor errors were predominantly omissions. The pilot provides descriptive evidence of feasibility and instrumentation readiness; larger controlled evaluation remains future work. This contribution is documented in Chapter 6, Section 6.3,

and Appendix 9.

Contributions 1 and 2 are supported by verified benchmark artifacts, automated tests, and repository evidence. Contribution 3 combines implemented infrastructure, a completed formative pilot study, and a documented path toward a larger controlled evaluation, which remains beyond the scope of this thesis.

1.6 Thesis Structure

Chapter 2 provides domain background on the F/A-18C cold-start procedure, the visual fact ontology, and DCS-BIOS telemetry, followed by a survey of related work in intelligent tutoring systems, simulator-based training, state-aware tutoring, VLM adaptation, and grounded AI assistance.

Chapter 3 presents the system architecture: design goals, ports-and-adapters layering, runtime data flow, telemetry processing, procedure and gating engines, replay infrastructure, and the VLM adaptation component.

Chapter 4 describes the telemetry grounding methodology, knowledge retrieval with BM25 and source policy, the seven-stage visual fact data pipeline, dataset curation and ontology evolution from 8-fact v1 to 13-fact v2, LoRA fine-tuning configuration for both backbones, and the benchmark protocol including metric definitions and holdout strategy.

Chapter 5 covers implementation-level details of the technology stack, core domain logic, telemetry integration, structured help generation, VLM visual fact extraction, and replay infrastructure.

Chapter 6 reports current technical results (VLM benchmarks, error analysis, runtime infrastructure readiness) in Part A, and reports the formative VR pilot study results in Part B.

Chapter 7 interprets the results, explains the evolution from the original proposal to the current system, draws design lessons from the two-generation ontology evolution, enumerates limitations, and outlines future work.

Chapter 8 summarises the completed work against the three contributions and research questions, and restates the scope boundaries within which the thesis should be evaluated.

Appendix 9 contains supplementary tables, full hyperparameter configurations, dataset statistics, CLI reference, and the 8-fact v1 baseline benchmark.

2 Background and Related Work

This chapter provides the domain and technical background necessary to understand the design of the proposed system, followed by a survey of related work across several intersecting research areas. Section 2.1 introduces the F/A-18C cold-start task that serves as the training scenario for the system. Section 2.2 describes the visual fact ontology, a structured intermediate representation that bridges raw cockpit screenshots and downstream procedural reasoning. The remainder of the chapter positions the present work against existing research in intelligent tutoring systems, simulator-based training, state-aware tutoring, vision-language model adaptation, and grounded AI assistance.

2.1 The F/A-18C Cold-Start Task

The procedural training scenario at the centre of this thesis is the F/A-18C cold-start, a standard pre-flight cockpit procedure that brings the aircraft from a powered-down state to a taxi-ready configuration. This task is canonical in DCS World training curricula and is representative of a broader class of fixed-sequence, state-sensitive procedural tasks in which the order of actions, the verification of intermediate system states, and the correct interpretation of cockpit instrumentation all contribute to successful completion.

2.1.1 Procedure Structure: S01–S25 and Phases P1–P6

The cold-start procedure is organised into 25 numbered steps (S01–S25), grouped into six coarse-grained phases (P1–P6) that reflect distinct operational stages:

P1 — Power-up and safety (S01–S03): Battery and generator activation, fire detection system tests, and auxiliary power unit (APU) start. These steps establish electrical power and confirm critical safety systems are functional before engine ignition.

P2 — Right engine start (S04–S07): Right engine crank, throttle advance at 25% RPM, bleed air system cycling, and caution/warning/advisory lights testing. The

right engine is started first to provide hydraulic and electrical power for subsequent procedures.

P3 — Displays and communications (S08–S09): Power-up of both Digital Display Indicators (DDIs), the Advanced Multipurpose Color Display (AMPCD), and the Head-Up Display (HUD); configuration of specific display pages (FCS page on left DDI, BIT root page on right DDI); and programming of COMM1 preset frequencies via the Up-Front Controller (UFC).

P4 — Left engine and core systems (S10–S14): Left engine start following verification of right-engine parameters in nominal ranges, INS alignment mode selection (ground or carrier), radar power-up, and OBOGS (On-Board Oxygen Generating System) activation.

P5 — FCS and flight controls (S15–S19): Flight Control System (FCS) reset and BIT (Built-In Test) execution, flap and trim configuration, and the "four-down" pre-flight checks covering the refuelling probe, speed brake, launch bar, and arrestor hook.

P6 — Pre-taxi instruments and final checks (S20–S25): Parking brake release, BINGO fuel setting, barometric and radar altimeter configuration, standby attitude indicator uncaging, and attitude source selection.

The procedure is strictly ordered: each step carries explicit precondition gates that depend on the completion of earlier steps and on the satisfaction of specific telemetry conditions. For example, S04 (engine crank right) requires that the APU start support has been established (S03 completion), while S10 (engine crank left) requires that right-engine parameters — RPM, temperature, fuel flow, nozzle position, and oil pressure — all fall within their nominal green ranges before the left engine is engaged. These inter-step dependencies make the cold-start task a compelling test case for context-aware tutoring: a learner who skips a verification or misjudges an instrument reading can propagate errors through the remainder of the sequence.

2.1.2 Cockpit Displays and the Composite Panel

The visual fact extraction subsystem (ğ2.2) operates on a fixed composite panel image that captures three cockpit display regions, arranged from top to bottom:

- **Left DDI (Digital Display Indicator):** A multi-function display used for the FCS page, SUPT (Setup) page, TAC (Tactical) page, and other system pages. During the cold-start, the left DDI is primarily used for FCS monitoring (S08, S15).
- **AMPCD (Advanced Multipurpose Color Display):** The central colour display, used for the HSI (Horizontal Situation Indicator) page, INS alignment status, map layers, and navigation data. In the cold-start, the AMPCD becomes critical during the INS alignment phase (S12).
- **Right DDI:** A second multi-function display, used during the cold-start for the BIT root page (S08) and later for the FCS-MC BIT sequence (S18). The right DDI is also where BIT failure indications and FCS BIT intermediate and final results appear.

The composite panel image consolidates these three regions into a single input for the vision-language model (VLM). This design choice avoids mismatches between training and inference (where the model would be trained on per-region crops but evaluated on a full composite, or vice versa) and simplifies the multi-turn visual pipeline to a single-image extraction step per observation window [6].

2.1.3 DCS-BIOS Telemetry

DCS-BIOS provides a real-time UDP-based telemetry stream from DCS World, exporting the state of cockpit switches, indicators, warning lights, gauges, and other instruments as structured data. The tutoring runtime receives these raw BIOS frames and passes them through a telemetry enrichment pipeline that normalises the raw values into stable runtime variables (e.g., `rpm_r`, `apu_ready`, `ins_mode`), applies delta sanitisation to suppress noise and jitter, and latches discrete state-completion flags for gate evaluation.

This telemetry feed is the primary grounding signal for the tutoring system. Unlike vision-only approaches, which must infer procedure state from pixel-level evidence alone, the tutoring runtime can cross-reference visual observations against deterministic telemetry state. Steps such as S01 (battery and generators), S05 (right throttle idle), and S13 (radar mode OPR) are directly observable via DCS-BIOS and require no visual inference. Other steps, such as S08 and S18, involve display-page state that is not directly exported via DCS-BIOS and therefore rely on the visual fact extraction pipeline

described below.

2.2 Visual Fact Ontology

A central design decision in the proposed system is the separation between visual perception and procedural reasoning. Rather than training a model to directly produce a tutor decision ("the learner should now press PB5"), the system uses an intermediate structured representation called *visual facts*. A visual fact is a discrete, auditable proposition about the visible state of one cockpit display region at a single point in time. Each fact is labelled with one of three states:

`seen`: The visual evidence described by the fact is clearly visible in the current cockpit image.

`not_seen`: The visual evidence is absent or not discernible.

`uncertain`: The image is blurry, occluded, incomplete, or otherwise insufficient for a confident determination.

These facts are consumed downstream by the procedure engine (ğ**??**), which combines them with telemetry state, gating rules, and knowledge retrieval to decide whether a procedure step is visually confirmed, still pending, or blocked. This layered architecture isolates the VLM's responsibility to structured visual evidence extraction, reserving procedure-level decisions for deterministic rule-based components.

2.2.1 From 8-Fact v1 to 13-Fact v2

The visual fact ontology evolved across two generations, reflecting lessons from early fine-tuning experiments and the needs of the runtime procedure pack.

8-fact v1 (early baseline). The first-generation ontology contained eight facts targeting the three display regions. These facts covered FCS page visibility (left DDI), BIT page and BIT failure visibility (right DDI), FCS-MC page and in-test status (right DDI), FCS BIT result visibility (right DDI), INS alignment page visibility (AMPCD), and the INS GO completion signal (AMPCD). This ontology was used to train the first LoRA adapter (LoRA-v1) on 180 human-reviewed images from Run-001, yielding promising in-distribution results but exposing a critical weakness: the `ins_go` fact mixed abstraction

levels by asking the VLM to recognise a high-level procedural completion state ("INS alignment is done") from a single visual frame. The result was a high false-positive rate on the independent Run-002 holdout set (13 of 50 samples predicted `seen` when the human label was `not_seen`).

13-fact v2 (runtime-aligned). The second-generation ontology addresses this by decomposing higher-level procedure states into lower-level, visually observable evidence. The current 13-fact set is defined in `vision_facts.yaml` and includes:

- **Left DDI facts:** `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_visible`.
- **Right DDI facts:** `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_resu` `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`.
- **AMPCD facts:** `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_vi` `ins_ok_text_visible`.

Note that `ins_go` from v1 has been decomposed into several lower-level AMPCD facts (`ins_grnd_alignment_text_visible`, `ins_ok_text_visible`, `hsi_map_layer_visible`) that each describe a specific, locally verifiable piece of visual evidence. The downstream procedure engine, rather than the VLM, is responsible for combining these atomic observations into a procedural judgement about whether the INS alignment phase is complete.

2.2.2 Sticky Facts and Step Bindings

Two additional mechanisms in the ontology improve its robustness for runtime use.

Sticky facts. Some visual facts represent transient states that, once observed, should not be overwritten by subsequent `not_seen` frames. For example, the `fcsmc_final_go_result_visibl` fact is marked as `sticky: true` with a time-to-live of 600 seconds. Once the VLM reports this fact as `seen`, it is latched in the runtime fact state and will not be overwritten by later `not_seen` or `uncertain` predictions within the TTL window. This prevents flickering and false negatives caused by display page changes, camera occlusions, or brief rendering artifacts that occur after the critical visual event has already been captured.

Non-sticky facts (e.g., `fcs_page_visible`, `bit_root_page_visible`) carry a shorter TTL of 2 seconds and are continuously re-evaluated, reflecting the expectation that these display pages may change as the learner navigates between different cockpit pages.

Step bindings. The vision facts are linked to procedure steps through explicit step bindings defined in `vision_facts.yaml`. A step binding specifies which facts must be `seen` for the procedure engine to consider the step visually confirmed. The current ontology defines two binding types:

- `all_of`: Every fact in the binding set must be `seen`. For example, S08 requires both `fcs_page_visible` (left DDI) and `bit_root_page_visible` (right DDI) to be confirmed.
- `any_of` (one-of fact collection): At least one fact in the set must be `seen`. This accommodates cases where a procedure state can be verified through any of several alternative visual cues.

Steps that rely on visual confirmation (S08, S15, S18) are registered as `vision_priority_steps` in the procedure pack, while steps directly observable through DCS-BIOS telemetry (S01, S03–S07, S09–S13, S21) are registered as `bios_priority_steps`. Steps that require manual confirmation or involve cockpit areas outside the three-display layout (S02, S14, S16–S20, S22–S25) are marked as `manual_or_out_of_layout_steps`. This tripartite classification ensures that each step's completion evidence is sourced from the most reliable available signal.

2.3 Intelligent Tutoring Systems and Procedural Training

Intelligent Tutoring Systems (ITS) have a long history in procedural training domains, from military equipment operation to medical procedure instruction. Classic ITS architectures typically model expert knowledge, track learner state through a student model, and deliver context-sensitive feedback through a tutoring module [5] (e.g., the classic four-component ITS model or a recent comprehensive survey).

In the procedural domain, constraint-based tutors and example-tracing tutors have been applied to tasks such as aircraft maintenance, programming, and equipment troubleshooting. Constraint-based and example-tracing tutors have been studied in procedural domains such as aircraft maintenance and programming, providing the theoretical foundation for tracking learner actions against a task model. These systems typically operate within closed, well-defined task environments where the space of correct and incorrect actions can be enumerated. The cold-start task studied in this thesis shares this property

of a closed procedure, but differs in two key respects. First, the system observes learner behaviour through simulator telemetry and cockpit visuals rather than through a custom tutor interface, making it applicable to unmodified simulation environments. Second, the tutoring output is generated dynamically by an LLM rather than retrieved from a pre-authored library of feedback messages, allowing the system to respond to arbitrary learner states within the procedure rather than only to anticipated error patterns.

More recent work has explored the use of large language models for tutoring and instructional dialogue. Prior work on LLM-based tutoring systems has demonstrated that large language models can produce fluent, contextually appropriate instructional text, but they typically operate in unstructured, conversational settings and do not enforce the strict evidence-grounding constraints that characterise the proposed system approach. In particular, the present work constrains every overlay target and step recommendation to have a verifiable evidence source — telemetry, a gating rule, a retrieved document snippet, or a visual fact — rather than relying solely on the model's parametric knowledge.

2.4 Simulator-Based Training and Instrumentation

High-fidelity simulation environments such as DCS World provide realistic, repeatable platforms for procedural training. The use of simulators in aviation training is well established, with extensive research on transfer of training from simulated to real aircraft, with extensive research on simulator-based training effectiveness in aviation.

A key challenge in simulator-based training research is instrumentation: the means by which a training system observes learner actions and cockpit state. Prior work has instrumented simulators through screen capture, event logging, or direct state export APIs (e.g., X-Plane data output, Microsoft Flight Simulator SimConnect, or community-developed telemetry bridges). The present work uses DCS-BIOS, a community-developed telemetry bridge that exports DCS World cockpit state via UDP. DCS-BIOS provides a richer signal than screen capture alone, exposing the exact position of switches, the numerical values of engine parameters, and the state of warning lights without requiring computer vision to recover them. This allows the system to reserve visual perception for display-page states (which are not exported by DCS-BIOS) while relying on deterministic telemetry for switch positions and gauge readings.

DCS World [2] serves as the simulation platform employed in this research; DCS-BIOS provides the telemetry interface used throughout the system.

2.5 Rule-Based and State-Aware Tutoring

The proposed system makes extensive use of explicit gating rules, deterministic step inference, and telemetry-driven state tracking alongside its LLM-based help generation. This hybrid design places it at the intersection of rule-based and neural approaches to intelligent assistance.

Rule-based approaches to procedure tracking — using finite-state machines, petri nets, or production rules to model task progress — offer strong guarantees of interpretability and correctness—for example, plan recognition for procedural tasks and finite-state models of procedure execution. These approaches, however, typically require manual authoring of procedure models and do not generalise to novel learner errors or unexpected state configurations. The proposed system uses manually authored procedure packs (defining steps, gates, and evidence requirements) as the backbone of its state tracking, but delegates the natural-language explanation and overlay target selection to the LLM. This division of labour preserves the interpretability of the procedure model while enabling the flexibility of language-model-generated feedback.

Hybrid neuro-symbolic architectures have been proposed for task guidance, combining neural perception or generation with symbolic task models (e.g., SayCan-style grounding, or LLM integration with symbolic planners). The present work differs from these efforts in its emphasis on evidence grounding: every actionable output of the system must be traceable to a specific, auditable evidence source (a telemetry variable, a visual fact observation, a gating rule evaluation, or a retrieved document chunk). This constraint is not merely an architectural preference but a safety requirement: for cockpit procedures, an incorrectly highlighted control or a misidentified procedure state could confuse or mislead the learner.

2.6 Foundation Models and VLMs for Situated Assistance

The adaptation of vision-language models (VLMs) to domain-specific structured extraction tasks is a rapidly growing area. Parameter-efficient fine-tuning methods, particularly Low-Rank Adaptation (LoRA), have made it feasible to adapt large pre-trained models to narrow domains with modest computational resources and small curated datasets. [3]; [4]; [1].

The VLM adaptation pipeline in this thesis (ğ**??**) follows a capture-prelabel-review-

train-benchmark loop that is notable for its use of human-reviewed small data (hundreds of images, not tens of thousands) to drive meaningful improvements in structured extraction accuracy. This contrasts with large-scale VLM fine-tuning efforts that rely on web-scale data and broad instruction tuning. The present work's use of hundreds of annotated samples rather than tens of thousands aligns with emerging interest in small-data domain adaptation and few-shot task-specific fine-tuning for vision-language models.

Prior work on visual understanding of graphical user interfaces and structured screen content has explored tasks such as widget detection, screen summarisation, and UI state classification (e.g., Screen2Words, Widget Captioning, and vision-based UI automation). The cockpit display domain studied here shares structural similarities with GUI understanding — fixed regions, structured layouts, symbolic content — but differs in the criticality of the task and the need for explicit, auditable evidence rather than open-ended descriptions.

The bilingual SFT strategy used in this work (training both English and Chinese variants from the same images and labels) is related to cross-lingual transfer and instruction diversity in fine-tuning. While cross-lingual instruction tuning and multilingual SFT for VLMs are active research areas, the primary motivation here is not multilingual deployment but rather instruction diversity as a form of regularisation: by training on two languages, the model is encouraged to attend to the visual content rather than to language-specific priors, while the structured JSON output format remains language-independent.

2.7 Explainable and Grounded AI Assistance in AR/VR

The original thesis proposal envisioned an in-headset augmented reality / virtual reality tutoring interface running on the Meta Quest 3. While the current implementation operates on a desktop DCS runtime (see ğ1.4), the long-term research agenda remains anchored in immersive procedural training.

Research on augmented reality guidance for procedural tasks has demonstrated the value of spatially registered instructions for assembly, maintenance, and repair tasks. Surveys of AR-guided procedural training cover applications in assembly, maintenance, and medical procedure guidance. These systems typically use pre-authored content (arrows, text, animations) registered to known physical objects. The proposed system's vision differs in that the instructional content is generated at runtime from the current

procedure state and visual evidence, rather than retrieved from a fixed library.

Work on explainable AI in decision-support systems has emphasised the importance of traceable reasoning, particularly in safety-critical domains such as aviation decision support and high-stakes recommendation systems. The evidence-grounding constraint in this system aligns with this principle: each overlay target and recommended step must carry an evidence reference that links the output to its source (a telemetry variable, a gating rule, a visual fact, or a retrieved document chunk). This audit trail is designed to support both runtime debugging (why did the system highlight this control?) and post-hoc analysis of help cycles.

Work on LLM-based assistance in VR/AR that specifically addresses grounding, evidence, or procedural correctness remains sparse compared with open-ended conversational assistance, motivating the evidence-grounding approach taken in this thesis.

2.8 Positioning of This Work

The proposed system occupies a position at the intersection of several research areas that are rarely combined in a single system: it applies a **telemetry-grounded** approach rather than a vision-only or text-only approach to procedural state tracking; it uses **parameter-efficient VLM adaptation** to produce structured visual facts rather than free-form descriptions; it enforces **explicit evidence grounding** on all actionable outputs; and it treats the **procedure pack** as a first-class configuration artifact that can be authored, versioned, and replayed independently of the model. The concrete contributions arising from this positioning are enumerated in Section 1.5.

The work presented in this thesis represents a technical baseline rather than a completed evaluation. The VLM adaptation results (ğ6) provide evidence that structured visual fact extraction can be measurably improved through small-data domain fine-tuning, but they do not yet constitute evidence of learning gains, workload reduction, or transfer. The human-subject evaluation envisioned in the original thesis proposal remains a planned next phase (ğ**??**), and the current system is best understood as the runtime and instrumentation infrastructure that makes that evaluation feasible.

3 System Design

This chapter presents the architecture of a telemetry-grounded tutoring runtime for procedural training. It covers the design rationale, the evidence-grounded help cycle, the core components, and the harness engineering that enforces correctness. Methodology-level detail—the VLM adaptation protocol, dataset curation, and benchmark design—is deferred to Chapter 4; implementation-level specifics appear in Chapter 5.

3.1 Design Goals and Requirements

The thesis addresses the problem of providing grounded, context-aware procedural guidance in the F/A-18C cold-start procedure—a fixed sequence of 25 core steps (S01–S25, extended to S33 in the current pack) across six operational phases. Each step has preconditions that must be satisfied before execution and completion conditions that must be met before advancing. From this problem, six design goals are derived:

G1 – Simulator-based procedural tutoring. The system must observe learner actions through simulator state, determine the current procedural step, and provide timely, grounded guidance. Tutoring output must reference concrete cockpit elements and cite evidence.

G2 – State-aware step guidance. Procedure progress is tracked through a formal state machine. Preconditions are evaluated before step activation; completion conditions are verified before advancement. Guidance must be conditional on current simulator state, not merely step order.

G3 – Separation of tutoring logic from simulator integration. The core tutoring logic—procedure tracking, gating, scoring, help generation—must be independent of any particular simulator or model provider. Simulator-specific code is isolated behind stable interfaces so the core can be tested without a running simulator.

G4 – Replayability and inspectability. Every interaction is recorded as structured, timestamped events supporting offline replay, automated scoring, and post-hoc analysis.

G5 – Model-optional operation with deterministic fallback. When a language model is available, it provides richer diagnoses; when unavailable or when its output fails validation, the system degrades gracefully to deterministic, rule-based behaviour. A VLM-based visual fact extraction path complements telemetry gating for display-page states not exported through DCS-BIOS.

G6 – Declarative procedure configuration. Procedure definitions, UI targets, telemetry mappings, error taxonomies, and vision fact ontologies are expressed as declarative YAML configuration (a *procedure pack*) rather than hard-coded in source code.

3.2 Architecture: Ports and Adapters

The system follows a ports-and-adapters (hexagonal) architecture organised into three layers with strict dependency direction: **adapters** $\rightarrow$ **ports** $\leftarrow$ **core**. The core never references adapter code directly.

Core layer. Contains all domain logic independent of any simulator, model provider, or external service: the procedure state machine, gating rule engine, scoring and interaction-metrics engines, vision fact ontology with sticky/TTL persistence, BM25 retrieval, dynamic JSON Schema generation for help responses, and security infrastructure.

Ports layer. Defines four stable Python protocols: `ModelPort` (reasoning), `KnowledgePort` (retrieval), `VisionPort` (visual input), and `TelemetryPort` (simulator state). These are the only dependency injection points between core and the outside world.

Adapters layer. Contains all simulator-specific, provider-specific, and infrastructure-specific code: DCS-BIOS telemetry receivers, telemetry enrichment and delta sanitisation, DCS overlay senders, OpenAI-compatible model clients (with and without multimodal support), Ollama fallback, a deterministic model stub, local BM25 knowledge adapter, and the vision fact extraction runtime.

Two further structural elements complete the architecture:

Procedure packs (`packs/`). A directory of declarative YAML files describing one procedural task: step list, precondition and completion gates, UI targets and element identifiers, telemetry variable mappings, error taxonomy, delta sanitisation policy, and vision fact ontology with step bindings. The current repository contains the `fa18c_startup` pack.

Runtime entrypoints. The `simtutor` package provides a command-line interface for simulation, replay, scoring, VLM recording, and validation. The live runtime module orchestrates the full loop: DCS-BIOS ingestion, telemetry enrichment, knowledge retrieval, model-based help generation, VLM visual fact extraction, overlay execution, and event logging.

The separation serves four purposes. **Testability**: core logic can be tested with mock adapters without DCS, a GPU, or network access. **Maintainability**: simulator API changes affect only the relevant adapter. **Transferability**: porting to a new simulator requires new adapters and packs; the core is reused as-is. **Independent evolution**: the VLM pipeline can be developed, trained, and benchmarked independently of the tutoring runtime.

3.3 The Evidence-Grounded Help Cycle

The central architectural mechanism is the evidence-grounded help cycle: a structured pipeline from simulator observation through tutoring output and logging, in which every actionable output must carry a traceable evidence reference.

3.3.1 Help Cycle Stages

Observation. The simulator adapter—a DCS-BIOS UDP receiver—produces a stream of timestamped observations containing raw simulator state. These are enriched through variable resolution and delta processing into normalised key–value state variables.

Tutor request. When the learner triggers help, a tutor request is created carrying the current observation and an assembled context: normalised state variables, sanitised recent deltas, candidate procedure steps, a deterministic step hint, an overlay target allowlist, BM25-retrieved knowledge snippets ($k = 5$), and, when available, the current vision fact summary.

Model inference. The assembled context is formatted into a prompt and sent to the language model, which returns a structured JSON object conforming to a dynamically generated HelpResponse JSON Schema.

Response validation and mapping. The raw model output passes through four validation stages: JSON extraction with minimal repair (code fence removal only), schema validation against the runtime-generated schema, allowlist checks on step identifiers and overlay targets, and evidence-grounding validation. Validated responses are mapped to overlay actions resolved through the UI map. Failed validation triggers deterministic fallback.

Action execution. Overlay actions (highlight, clear, pulse) are validated against the UI target allowlist and dispatched to the simulator. Auto-click and control-injection are forbidden.

Event logging. Every stage is recorded as a versioned, timestamped event in a JSONL event log—the authoritative record for all downstream analysis, replay, and scoring.

3.3.2 Schema-Level Evidence Grounding

The evidence constraint operates at the schema level, not as a post-hoc check. The HelpResponse JSON Schema is generated dynamically from the procedure pack and UI map at runtime. For every overlay target in the schema's `targets` array, a conditional `if/then` rule requires that the accompanying `evidence` array contain at least one item referencing that target. Each evidence item must declare a `type` from five grounding kinds—telemetry variable (`var`), gating rule (`gate`), knowledge snippet (`rag`), recent state change (`delta`), or vision fact (`visual`)—each with a corresponding `ref` prefix. An LLM response that proposes an overlay without a matching, validly-typed evidence item is structurally invalid and rejected before any action reaches the simulator.

This mechanism produces a unified audit trail. For example, highlighting the `battery_switch` must be accompanied by an evidence item such as `{"target": "battery_switch", "type": "var", "ref": "VARS.battery_switch", "quote": "0"}`, tracing the action to the telemetry variable that justified it. The five evidence types are validated against the keys actually present in the help-cycle context; fabricated references are rejected.

3.4 Telemetry Processing

The telemetry processing subsystem transforms raw DCS-BIOS data into stable state representation through four stages:

1. **Raw ingestion.** The DCS-BIOS receiver decodes binary UDP export frames into structured JSON with wall-clock timestamps and monotonic sequence numbers.
2. **Variable resolution.** A telemetry map (declared in the procedure pack) projects raw BIOS fields onto normalised state variables. The resolver tracks unavailable sources in a `vars_source_missing` set, enabling downstream components to distinguish "value is zero" from "value is unknown"—a distinction critical for correct gating behaviour.
3. **Delta sanitisation.** A configurable policy (per-variable thresholds, debounce windows, filtering rules) suppresses spurious changes from switch bounce, sensor drift, and UDP out-of-order delivery.
4. **Delta aggregation.** Sanitised deltas are accumulated over a configurable window and compressed into a compact prompt-safe summary.

The telemetry pipeline runs continuously, independent of any help cycle.

3.5 Procedure Engine and Gating Rules

3.5.1 Procedure State Machine

The procedure engine maintains a finite-state machine over the ordered step sequence. Each step is in one of four states: `pending`, `active` (at most one), `done`, or `blocked`. All transitions emit internal event records with ISO-8601 timestamps.

3.5.2 Gating Rule Engine

Precondition and completion gates are evaluated using a DSL with four operators: `var_gte` (threshold), `arg_in_range` (interval), `flag_true` (boolean), and `time_since` (elapsed time). The evaluator includes explicit unknown-value handling: missing, source-missing, or sentinel-value variables cause rule failure with a diagnostic reason. Rules short-circuit on first failure; the failure index and reason feed into the deterministic step hint.

3.5.3 Deterministic Fallback

When no model is available or model output fails validation, the system actively computes a useful step hint through deterministic inference. The fallback analyses three information sources: (1) the active step and its unsatisfied gating rules, (2) the learner's recent UI interactions, and (3) observability metadata that classifies each missing evidence source as a *hard blocker* (gate explicitly fails) or *soft blocker* (telemetry source absent, true state unknown). This distinction prevents the tutor from incorrectly claiming a step is unsatisfied when the evidence is simply unavailable. The fallback produces a text-only hint restricted to the active step; overlay highlights are suppressed unless a local rule can uniquely determine a valid target.

3.5.4 Step Classification by Evidence Source

Each step is classified by its primary evidence source, declared in the procedure pack:

- **BIOS-priority steps** (S01, S03–S07, S09–S13, S21): verified primarily through DCS-BIOS telemetry.
- **Vision-priority steps** (S02, S08, S14–S20, S22–S25): verified through VLM visual fact extraction from cockpit displays.
- **Manual/out-of-layout steps**: steps requiring manual confirmation or involving cockpit areas outside the three-display composite panel.

This tripartite classification ensures each step's completion evidence is sourced from the most reliable available signal.

3.6 Harness Engineering and Evidence Validation

A distinguishing architectural feature of the system is the *step harness*: a validation layer that sits between the help-cycle response generation and the overlay action execution, enforcing that every actionable output satisfies evidence-grounding, allowlist, and procedural consistency constraints before reaching the simulator.

3.6.1 Harness Architecture

The harness operates in four stages applied sequentially to every candidate tutor response:

1. **Spec derivation.** For each active procedure step, the harness derives a *step harness spec* from the procedure pack: the set of permitted overlay targets (from the UI map), the required evidence categories (from the step's gating configuration and observability metadata), and the banned target categories (e.g., targets belonging to a different procedure phase or requiring visual confirmation when the vision path is unavailable).

2. **Decision adjudication.** The harness evaluates the model's proposed diagnosis and next-step recommendations against the current procedure state. A `HarnessDecision` dataclass records whether each proposed action is `accepted`, `repaired` (modified to align with procedure state), or `rejected` (blocked as unsafe or inconsistent). Repairs are traceable: every modification carries a reason code referencing the specific harness rule that triggered it.

3. **Evidence snapshot trace.** Before any overlay action is executed, the harness captures an *evidence snapshot*—a frozen record of the telemetry variables, gating rule evaluations, knowledge snippet references, and vision fact states that jointly justify the current overlay plan. This snapshot is embedded in the event log alongside the tutor response, creating a verifiable link between the evidence that was available at decision time and the action that was taken.

4. **Coverage matrix.** A regression coverage matrix maps each procedure step to a set of harness test scenarios (fixture-based replay cases), each exercising a specific harness rule: correct advancement, safe rejection of wrong-target proposals, repair of incomplete visual evidence, handling of moving/settling controls, and prevention of VLM evidence leakage into non-visual steps.

3.6.2 Fixture-Based Regression Testing

The harness is validated through a suite of *live help fixtures*: recorded telemetry snapshots captured at specific procedure states that previously exposed harness failures. Each fixture replays a specific state through the full help cycle and asserts that the harness produces the expected outcome (correct target, correct step, or correct rejection). As of the current implementation, the regression suite covers 20 live-fixture regression entries in the coverage matrix spanning steps S01 through S33, including:

- Correct advancement when completion conditions are satisfied (telemetry-based and vision-based);

- Safe rejection of overlay targets that violate the UI allowlist or belong to a different procedure phase;
- Repair of wrong-target proposals (e.g., PB18 repaired to PB5 for BIT root substate);
- Prevention of VLM evidence leakage into non-visual steps (stale S08 visual facts must not drive S09/S10 help);
- Correct handling of moving/settling controls (refuelling probe, arresting hook) that require wait guidance rather than repeated prompts;
- Consistent interaction semantics (left-click vs. right-click, mouse wheel for rotary controls).

The harness coverage matrix and fixture suite serve a dual role: they are both a development-time regression safety net and an existence proof that systematic evidence-grounding validation can be enforced at the architectural level rather than relying on model output trustworthiness.

3.7 Simulator Adapter Layer

The simulator adapter layer isolates DCS-specific code behind the port interfaces. Three adapters are defined:

Telemetry input. Receives DCS-BIOS state data over UDP and feeds the telemetry enrichment pipeline.

Overlay output. Sends highlight, clear, and pulse commands to DCS via UDP. A Lua script inside DCS renders graphical overlays on cockpit elements. The adapter maps abstract target names to DCS-internal point codes using the pack's UI map.

Vision input. Triggers screenshot captures in DCS. The Lua-side integration renders the three primary cockpit displays into a single composite image; the Python side manages frame selection and passes images to the VLM extractor.

The VR code path has been implemented (VR_MIRROR configuration, tutor text UDP injection, composite-panel capture compatibility in VR mode) and was exercised in a formative pilot study (Section 6.3); larger controlled human-subject experiments remain future work.

3.8 Event Logging and Replay

All runtime occurrences are represented as versioned, timestamped `Event` objects with a fixed schema and persisted in JSONL format via an append-only store with immediate flush after each write, ensuring crash-safe logging.

A recorded event log can be replayed offline to verify step ordering and state machine consistency. The replay evaluation infrastructure supports suite-driven regression replay: a YAML suite file specifies a pack path, test cases with expected outcomes, and optional vision frame directories. Five replay evaluation cases cover distinct scenario types (no-operation, battery-on, battery-and-generator-off, FCS reset and BIT, INS alignment), each with DCS-BIOS raw captures and per-frame sidecar files.

The scoring engine counts omissions and state violations with configurable per-category weights. Interaction metrics characterise learner behaviour: stall time, help requests, LLM triggers, UI click counts, and overlay request/acknowledgement counts.

3.8.1 Current Limitations

The replay infrastructure has been exercised with synthetic regression scenarios and fixture-based replay at the telemetry level, including cases with vision frame sidecars. End-to-end replay of full multimodal sessions with synchronised vision frames from live human-in-the-loop runs has not yet been validated.

3.9 VLM Adaptation for Visual State Understanding

The VLM component extracts structured *visual facts* from cockpit screenshots—intermediate propositions about visible display content. Visual facts are not final tutoring decisions; they provide visual evidence that the tutoring runtime combines with telemetry, procedure state, and documentation. This layered architecture isolates the VLM's responsibility to structured visual evidence extraction, reserving procedure-level decisions for deterministic rule-based components.

3.9.1 Vision Fact Ontology (13-Fact v2)

The current ontology defines 13 facts spanning three display regions:

- **Left DDI**: `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_v`

- **Right DDI**: `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_result_vi` `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`.
- **AMPCD**: `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`.

Each fact has a three-valued state (`seen`, `not_seen`, `uncertain`), a time-to-live, an optional `sticky` flag (once `seen`, not overwritten by subsequent `not_seen` within TTL), and step bindings via `all_of` / `any_of` conditions. The evolution from the 8-fact v1 ontology to the current 13-fact v2 design is a central methodological and evaluation narrative, detailed in Chapter 4 and evaluated in Chapter 6.

3.9.2 VLM Adaptation Pipeline

The VLM used at runtime is fine-tuned through a seven-stage pipeline spanning screenshot capture, AI pre-labeling, human review in Label Studio, bilingual supervised fine-tuning export, LoRA fine-tuning via Unsloth + PEFT + TRL, and automated base-vs-LoRA benchmarking on independent holdout sets with verified zero training overlap. Full methodological detail appears in Chapter 4.

3.9.3 Runtime Integration

When a help request arrives and the active step is vision-priority, a capture trigger is sent to DCS. Pre-trigger and trigger frames are rendered and sent to the VLM. The returned facts are merged into the vision fact snapshot (with sticky and TTL semantics) and included in the help-cycle context. If the vision adapter is unavailable or the multimodal request fails, the help cycle proceeds with telemetry and knowledge only. This graceful degradation is a deliberate design property: visual perception is an optional enhancement to the telemetry-grounded core, not a dependency.

4 Methodology

This chapter describes the methodological framework for the telemetry-grounded tutoring runtime and the VLM adaptation pipeline. Section 4.1 covers telemetry processing and gating rule evaluation. Section 4.2 describes knowledge grounding via BM25 retrieval with source policy. Section 4.3 presents the seven-stage visual fact data pipeline. Section 4.4 details dataset curation and the ontology evolution from 8-fact v1 to 13-fact v2. Section 4.5 specifies the LoRA fine-tuning configuration for three backbone models under matched data recipe. Section 4.6 defines the benchmark protocol, metrics, and holdout strategy.

4.1 Procedure Runtime and Telemetry Grounding

The procedure runtime ingests raw simulator telemetry, normalises it into stable state variables, evaluates gating rules to determine step eligibility, and provides the resulting state representation to the help-cycle context assembler.

4.1.1 Variable Resolution and Source-Missing Tracking

A telemetry map (declared in the procedure pack) defines how raw DCS-BIOS fields are projected onto normalised state variables. The variable resolver produces a flat dictionary of key–value pairs and tracks unavailable or unknown sources in a `vars_source_missing` set, enabling downstream components to distinguish "the value is zero" from "the value is unknown"—a distinction essential for correct gating rule behaviour. A missing variable never silently evaluates to a successful condition.

4.1.2 Gating Rule Evaluation

The procedure engine evaluates two gate types per step: precondition gates (before activation) and completion gates (before marking done). The DSL supports four operators: `var_gte` (threshold), `arg_in_range` (interval), `flag_true` (boolean), and `time_since`

(elapsed time). The evaluator includes explicit unknown-value handling: missing, source-missing, or sentinel-value variables cause rule failure with a diagnostic reason. Rules short-circuit on first failure; the failure index and reason propagate to the deterministic step hint and help-cycle context.

4.1.3 Delta Sanitisation and Aggregation

Raw telemetry variables change at high frequency due to simulator physics even without learner action. Two processing stages produce a stable representation: delta sanitisation suppresses spurious changes via per-variable thresholds, debounce windows, and filtering rules; delta aggregation accumulates sanitised deltas over a configurable window and compresses them into a prompt-safe summary included in the help-cycle context.

4.1.4 Deterministic Fallback

When no language model is available or model output fails validation, the procedure runtime falls back to deterministic step inference. The fallback examines the active step, its unsatisfied gating rules, and recent UI targets to produce a text hint restricted to the active step. Overlay highlights are suppressed unless a local rule can prove a unique, valid target. The mechanism distinguishes hard blockers (gate explicitly fails) from soft blockers (telemetry source absent), preventing incorrect claims that a step is unsatisfied when evidence is simply unavailable.

4.2 Knowledge Grounding and Source Policy

Knowledge retrieval uses BM25, a bag-of-words ranking function from the probabilistic information retrieval family. Documents are indexed at the sentence level from the F/A-18C NATOPS flight manual and supplemental cold-start documentation. At retrieval time, a grounding query is constructed from the active step identifier, its description, and the current telemetry observation. The query is scored against indexed sentences using standard BM25 weighting ($k_1 = 1.2$, $b = 0.75$); the top-k scoring snippets ($k = 5$) are included as evidence context in the help-cycle prompt.

A declarative source policy enforces an allowlist of permitted document sources. Each indexed sentence carries a source identifier; only snippets from allowed sources are included in retrieval results. This prevents the model from being exposed to procedure variants that contradict the current pack configuration (e.g., carrier-based procedures

when the current profile is airfield) and bounds the retrieval scope to curated, verified documentation.

4.3 Visual Fact Data Pipeline

The visual fact data pipeline transforms raw DCS cockpit screenshots into structured visual fact extraction models through seven stages. Each stage produces versioned, inspectable artifacts, enabling full traceability from raw image to benchmark metric.

1. **Screenshot capture.** DCS renders the three primary multifunction displays into a single composite-panel image. Captures are triggered at configurable intervals and stored alongside a sidecar manifest. The composite format keeps training, pre-labeling, evaluation, and runtime inference on the same input distribution.

2. **VLM pre-labeling.** A large VLM (Qwen 397B-class, accessed via OpenAI-compatible API) produces initial per-fact labels and evidence notes for each image. Pre-labeling seeds annotations at scale; human review corrects errors in the next stage.

3. **Human review in Label Studio.** Pre-labeled samples are imported into Label Studio. Each task presents the composite image, predicted labels, and a summary field. The reviewer corrects mislabeled facts and provides evidence notes. All reviewed samples carry an audit trail linking back to the raw image, the original pre-label, and the reviewer's corrections.

4. **Reviewed JSONL export.** Label Studio exports corrected labels as a JSONL file—the authoritative ground-truth artifact for all downstream stages.

5. **Bilingual SFT JSONL generation.** Each reviewed sample produces two training rows—one English, one Chinese—sharing the same image and fact labels but differing in instruction text. Each row is a three-message conversation: system message (JSON-only instruction), user message (image + fact definitions + decision boundaries), and assistant message (ground-truth JSON with `facts` array and `summary`). External metadata fields (`frame_id`, `session_id`) are excluded from the training target.

6. **LoRA fine-tuning.** The SFT JSONL files are used to fine-tune a base VLM via Low-Rank Adaptation (LoRA) using Unsloth (4-bit loading, LoRA injection, vis-

ion batch collation), PEFT (adapter definition), and TRL `SFTTrainer` (supervised fine-tuning loop). Adapter weights are saved separately from the base model.

7. **Base-vs-LoRA benchmark.** The fine-tuned LoRA adapter is evaluated against the unadapted base model on independent holdout sets. The benchmark script loads each model configuration in 4-bit inference mode, sends each sample through the same prompt used during training, validates output against the expected JSON schema, and computes metrics comparing predictions to human-reviewed ground-truth labels.

A defining methodological property is end-to-end traceability: every benchmark prediction can be traced back through training data, human-reviewed labels, VLM pre-labels, and the original DCS screenshot. The pipeline produces versioned artifacts at each stage, enabling rigorous audit from individual fact misclassification to aggregate benchmark metrics.

4.4 Dataset Curation

4.4.1 Dataset Overview

The visual fact data pipeline produced six datasets over successive capture and review cycles. Table 4.1 summarises each dataset.

Table 4.1: Dataset inventory. All counts from `stats.json` files.

Dataset	Role	Samples	Ontology	Reviewe
Run-001 (`vision_sft`)	Training source	180	8-fact v1	180
Run-002 (`vision_sft_holdout_run002`)	Holdout	50	8-fact v1	50
Run-002 newfacts	Holdout	50	13-fact v2	50
Run-003 (`vision_sft_run003`)	Training source	220	13-fact v2	220
Run-004 (`vision_sft_holdout_run004_random`)	Holdout	100	13-fact v2	100
Run-005 (`composition_rebalance`)	Training supplement	122	13-fact v2	122

Run-003 and Run-005 contain samples where the human reviewer left the summary field blank (82 and 25 samples, respectively). These samples remain valid for training and evaluation because the facts array—the primary supervised target—is fully annotated for every fact in every sample.

4.4.2 Ontology Evolution: 8-Fact v1 to 13-Fact v2

The initial 8-fact v1 ontology defined eight facts spanning three display regions. Two structural weaknesses were identified during early benchmark analysis. First, it mixed abstraction levels: some facts described low-level visual observations (`fcs_page_visible`), while others encoded higher-level procedural judgements (`ins_go`, representing an INS alignment completion signal rather than a directly observable visual feature). Second, several facts shared overlapping semantics: `right_ddi_fcsmc_page_visible` conflated region identifiers with fact semantics, and the compound `ins_go` fact asked a single-frame VLM to make a procedural completion judgement that conflated perception with reasoning.

The 13-fact v2 ontology, used in Run-003 through Run-005 and in the current runtime, addressed these weaknesses through three strategies:

1. **Decomposition of compound targets.** The ambiguous `ins_go` was decomposed into two lower-level, directly observable facts: `ins_grnd_alignment_text_visible` (requiring literal GRND alignment block text) and `ins_ok_text_visible` (requiring clearly readable OK text near the alignment block). The FCS-MC state sequence was decomposed from a single `fcs_bit_result_visible` into a four-state chain (`fcsmc_page_visible`, `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`).

2. **Region-independent naming.** Region identifiers were removed from fact names (`right_ddi_fcsmc_page_visible` became `fcsmc_page_visible`); page-visibility facts were added for TAC and SUPT menus to cover the left DDI; HSI-related facts were split into `hsi_page_visible` and `hsi_map_layer_visible`.

3. **Abstraction-level discipline.** Every v2 fact is a locally verifiable visual proposition evaluable from a single cockpit image without procedural context. The downstream procedure engine—not the VLM—combines atomic observations into procedural judgements using telemetry, phase, and timing information.

4.4.3 Training Recipe: Run-003 + Run-005×2

The primary training recipe combines Run-003 (220 reviewed samples, bilingual export: 220 EN + 220 ZH = 440 rows) with Run-005 composition rebalance data included twice (122 reviewed samples, bilingual export: $122 \times 2 \times 2 = 488$ rows), yielding a combined training set of **928 rows**.

Run-005 was captured specifically to address class imbalance in Run-003, where several critical facts had extremely low `seen` counts: `ins_ok_text_visible` (12 of 220), `fcs_page_x_marks_visible` (16), and the three FCS-MC sub-states (18 each). Run-005 targeted cockpit states in which these rare facts are `seen`; double-counting (Run-005×2) further amplifies their representation. No internal evaluation split was used (`eval_rows = 0`); all evaluation is performed on independent holdout datasets.

4.4.4 Holdout Independence

Two holdout sets were constructed for the 13-fact v2 evaluation: Run-002 newfacts (50 samples, re-annotated from the Run-002 capture session) and Run-004 random (100 samples, uniformly random cockpit state sampling). Both are distinct DCS capture sessions with verified zero exact overlap with any training data—the overlap check compared image artifact paths and sample identifiers across training and holdout splits.

In contrast, the Run-001 benchmark used in early 8-fact v1 experiments is a **contaminated development set**: the evaluation data pool overlaps with the training data pool. Run-001 results are reported for regression analysis only (see Appendix 9) and are explicitly distinguished from independent holdout results.

4.5 LoRA Fine-Tuning Configuration

4.5.1 Backbone Models and Comparison Strategy

Three VLM backbones were fine-tuned under the matched Run-003 + Run-005×2 data recipe (928 bilingual rows, 13-fact v2 ontology), organised in three comparison tiers:

Primary — Qwen3.5-9B. A 9-billion-parameter VLM (`Qwen/Qwen3.5-9B-Base`). This is the primary experimental backbone, selected for its balance of capability and training efficiency. LoRA adapters are applied to both vision and language layers.

Scale/version comparison — Qwen3.6-27B. A 27-billion-parameter VLM from the same model family (`Qwen/Qwen3.6-27B`). This comparison examines the combined scale/version effect within the Qwen family under a matched data recipe and holdout protocol. The base model achieves substantially higher base-model performance (fact accuracy 0.91 on Run-002 newfacts, vs. 0.87 for Qwen3.5-9B), but the LoRA improvement margin is narrower (fact accuracy $\Delta = +0.077$ vs. $+0.123$ for

Qwen3.5-9B), consistent with the expectation that larger base models have less adaptation headroom under small-data fine-tuning.

Cross-family — Gemma-4-31B. A 31-billion-parameter VLM from Google DeepMind (`google/gemma-4-31B`). This comparison tests whether the data recipe generalises across model families. LoRA is restricted to language-only linear modules (`all-linear`; vision layers are not fine-tuned).

4.5.2 Training Stack and Hyperparameters

All three backbones share the same training stack: Unsloth (4-bit model loading, LoRA injection, vision batch collation), PEFT (LoRA adapter definition), and TRL `SFTTrainer` (supervised fine-tuning loop). Hyperparameters are matched wherever architecturally possible: learning rate 2×10^{-4}, LoRA rank $r = 16$, $\alpha = 16$, dropout $= 0.0$, 4 epochs, effective batch size 4 (batch size 1 $\times$ gradient accumulation 4), maximum sequence length 4096, seed 3407, warmup ratio 0.1, weight decay 0.01. Backbone-specific differences—LoRA target modules, vision layer fine-tuning, chat template, and GPU memory utilisation—are documented in Appendix 9, Table 9.1.

The epoch count of 4 was selected as a compromise for small-data domain adaptation: with 928 training examples, fewer epochs may not reliably teach the fixed JSON schema and cockpit fact boundaries, while more epochs risk overfitting to the training sessions. LoRA rank $r = 16$ provides moderate adaptation capacity—sufficient for cockpit layout patterns and JSON formatting without the overfitting risk of high-capacity adapters ($r \geq 64$). Dropout was set to zero for these first adaptation experiments, prioritising learnability over regularisation.

4.5.3 Bilingual SFT and Composition Rebalance

The bilingual SFT strategy produces two training rows per reviewed sample (English and Chinese instructions, identical image and ground-truth labels). This increases instruction-following robustness without additional annotation effort, but does not substitute for additional cockpit-state screenshots or hard-negative visual data.

The composition rebalance strategy (Run-005) addresses the class imbalance observed in Run-003. Several critical facts had extremely low positive counts: `ins_ok_text_visible` (12 of 220, 5.5%), `fcs_page_x_marks_visible` (16, 7.3%), and the three FCS-MC substates (18 each, 8.2%). Run-005 captured 122 additional samples targeting cockpit states where these rare facts are `seen`, improving combined coverage substantially (e.g.,

`supt_page_visible` from 20 to 54 combined seen; `hsi_page_visible` from 106 to 215; `ins_grnd_alignment_text_visible` from 58 to 130). Double-counting Run-005 in the recipe further amplifies the signal.

4.6 Benchmark Protocol

4.6.1 Metrics

The benchmark compares model predictions against human-reviewed ground-truth labels using the following metrics:

`json_valid_rate` Proportion of samples with parseable JSON output. Non-JSON responses (markdown, commentary, malformed braces) count as failures.

`schema_valid_rate` Proportion of samples whose parsed JSON conforms to the expected schema (correct keys, all 13 fact IDs present exactly once, all states in `{seen, not_seen, uncertain}`).

`fact_accuracy` Three-class accuracy over all facts in all samples.

`macro_f1` Macro-averaged F1 across three states, per fact, averaged over 13 facts.

`seen_f1` F1 for the positive `seen` class, per fact, averaged over 13 facts. This is a key metric because missing or hallucinating a `seen` state directly affects downstream procedure-state reasoning.

`sample_exact_match` Proportion of samples where all 13 facts match ground truth exactly.

`critical_false_positive_count` Count of errors where any of 6 critical facts (`fcs_page_x_marks_v` `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_vi` `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`) is predicted `seen` when ground truth is `not_seen` or `uncertain`. These are tracked separately because they represent the most consequential error type: incorrectly reporting a completion-critical state as observed could cause the tutoring system to prematurely advance the procedure.

4.6.2 Base vs. LoRA Comparison

Both configurations receive identical prompts, images, and generation parameters: temperature = 0.0 (greedy decoding), `max_new_tokens` = 1024, seed = 3407. The benchmark runs each sample independently, serialises predictions to JSONL, and computes metrics without post-hoc filtering. A comparison artifact (`comparison.json`) computes delta metrics (LoRA − Base) and an automated recommendation flag.

4.6.3 Benchmark Categories

Each benchmark run is assigned a `benchmark_kind`: `contaminated_dev_set` (evaluation data overlaps with training data pool; used for Run-001 regression only) or `heldout_new_session` (independent capture session with zero training overlap; used for Run-002 newfacts and Run-004 random). This distinction is maintained rigorously throughout the benchmark infrastructure and evaluation chapter.

4.6.4 Benchmark Artifacts

Each benchmark run produces a standardised directory: top-level summary with comparison deltas, per-model metric dictionaries with per-fact scores and confusion matrices, per-sample prediction and error JSONL files, a CSV table of per-fact per-model metrics, a human-readable markdown summary, and automatically generated charts (overall accuracy, per-fact macro F1, per-fact seen F1, critical false positives, and per-fact confusion matrices for both base and LoRA models). These artifacts constitute the full evidence chain for the technical results reported in Chapter 6.

4.7 Human-Subject Pilot Methodology

A small formative VR pilot study was conducted to gather descriptive evidence on how the tutoring system supports learners during the F/A-18C cold-start procedure compared with unaided practice. The pilot was designed as a preliminary field trial rather than a powered controlled experiment.

4.7.1 Participants and Conditions

Nine participants (P01–P09) were recruited from a convenience sample of DCS users with varied experience levels. Each participant completed one trial in a between-subjects

design with two conditions:

With-tutor (n=4). P04, P05, P08, P09. Participants received the full telemetry-grounded tutoring system with overlay highlights, VLM visual fact extraction, and structured help text displayed in the DCS in-game text area. Help was triggered via hotkey.

Without-tutor (n=5). P01, P02, P03, P06, P07. Participants received no tutor assistance. Telemetry was recorded passively for later step inference and error coding, but no overlays or help text were shown.

Assignment to conditions was not randomised; participants were assigned based on scheduling availability. Prior DCS and VR experience varied across participants (see Appendix 9, Table 9.5).

4.7.2 Task and Apparatus

All trials used the F/A-18C cold-start procedure with the full 33-step pack (S01–S33) in DCS World on a Meta Quest 3 via Quest Link. The composite-panel layout (left DDI, AMPCD, right DDI) was rendered on an external monitor for experimenter observation. The tutoring runtime ran on a Windows PC with remote model inference via an OpenAI-compatible endpoint. Each trial ended when the participant either completed the procedure (confirmed by terminal step completion or tutor declaration) or stopped voluntarily after being unable to progress.

4.7.3 Data Sources and Export Pipeline

For each trial, the experiment export pipeline produced a standardised set of artifacts:

- `trial_summary.csv`: per-trial aggregate metrics (completion status, observed duration, help requests, LLM triggers, VLM calls, overlay executed/rejected, fallback count, step completion counts).
- `step_coding.csv`: per-step coding with auto-coded errors (OM/CO/OR/PA/SV), evidence references, auto-confidence, and human review fields.
- `help_cycles.csv`: per-help-cycle detail (generation mode, vision status, VLM call status, overlay targets, fallback reason) for tutor-condition trials.

- `action_timeline.csv`: timestamped telemetry actions with fused step inference.
- `quality_gate.json`: configuration hash verification, event count, and metadata completeness check.
- `session.json` / `summary.json`: per-trial session metadata and interaction summary.

4.7.4 Error Taxonomy and Coding Policy

All steps were coded under a five-category error taxonomy: **OM** (omission — step not completed), **CO** (commission — step executed incorrectly or out of order), **OR** (order error), **PA** (prompting/assistance error — step completed only after tutor intervention or with incorrect tutor guidance), and **SV** (state violation).

The automated coding pipeline produced initial error labels (Auto_Error_* columns), which were then reviewed and corrected by a human coder (CodexTelemetryReview) using raw telemetry, help-cycle logs, and action timeline evidence. Human-coded columns (Error_OM through Error_SV) are the authoritative data source for all reported results.

4.7.5 Data-Quality Notes

Two trials require explicit data-quality documentation:

P08 — auto-summary override. The automated trial summary reported P08 as incomplete (Completed=no, TotalStepsCompleted=5/33). However, manual telemetry review confirmed that all 33 steps reached their required states. The auto-pipeline failed because it required explicit step evidence and gate completion signals that the passive visual inference path could not reconstruct for several vision-priority steps. Following the coding policy, manual coding overrides the auto-summary; P08 is reported as completed (33/33 steps) with zero manual errors. The pre-review coding file (`step_coding.before_codex_log_review.csv`) is preserved for audit in the experiment artifacts.

P02 — questionnaire metadata unresolved. P02's trial directory contains a questionnaire JSON file with internal participant_id = P01, indicating a file mislabelling during export. P02's task metrics (steps completed, errors) are derived from `trial_summary.csv`

and `step_coding.csv` and are usable. P02 is excluded from any workload, TLX, or confidence summaries because the questionnaire data cannot be confidently attributed.

4.7.6 Analysis Approach

Given the small sample ($n = 9$) and non-random assignment, the analysis is strictly descriptive. No null-hypothesis significance tests are performed. Results are reported as per-participant counts and condition-level summaries. Observed trial durations are reported but cannot be interpreted as completion time comparisons because without-tutor trials were stopped at different points by participants who could not progress further. The analysis focuses on three descriptive questions: (1) whether participants completed the procedure, (2) what categories of procedural error occurred and how they differed between conditions, and (3) how tutor interaction was used (help requests, VLM calls, overlay actions, fallback events).

4.8 Summary

The methodology is characterised by four properties. First, *end-to-end traceability*: every benchmark metric can be traced through training data, human-reviewed labels, and original DCS screenshots. Second, *strict holdout independence*: contaminated development benchmarks and independent holdout benchmarks are explicitly distinguished and interpreted differently. Third, *matched comparison design*: the identical data recipe, benchmark protocol, and evaluation sets are applied across three backbones (Qwen3.5-9B, Qwen3.6-27B, Gemma-4-31B), enabling controlled comparison of model scale, version, and family effects while holding data and evaluation constant. Fourth, *descriptive pilot analysis*: the human-subject pilot is analysed descriptively with explicit data-quality documentation and no inferential statistical claims.

5 Implementation

This chapter covers implementation-level specifics that directly support the research questions, focusing on the evidence-grounding enforcement mechanism, the deterministic fallback path, the VLM runtime integration, and the replay infrastructure. Implementation details not essential to the core thesis argument—CLI command reference, complete technology stack versions, Lua script inventory, and full hyperparameter configuration—are documented in Appendix 9.

5.1 Technology Stack

The Python codebase uses Python 3.10+ with Poetry for dependency management. Core library dependencies are minimal: `httpx` for HTTP, `PyYAML` for declarative configuration, `jsonschema` (Draft 2020-12) for validation of model outputs and telemetry frames, and `Pillow` for image handling in the VLM pipeline. Testing uses `pytest` with branch coverage.

Model serving is provider-agnostic. The primary path uses an OpenAI-compatible API endpoint (configured for vLLM, llama.cpp, or TGI servers). An Ollama fallback provides local compatibility. A deterministic stub returns fixed responses for testing. Multimodal support is toggled via configuration.

The DCS Lua-side integration comprises scripts for state export (DCS-BIOS protocol), overlay rendering (highlight, clear, and pulse commands via DCS's cockpit highlight API), and composite-panel screenshot capture. LoRA fine-tuning uses Unsloth with PEFT and TRL; pre-labeling and human review are conducted in Label Studio.

5.2 Evidence Grounding Enforcement

The evidence-grounding mechanism described in Section 3.3 is implemented through three coordinated components.

Dynamic JSON Schema generation. The function `build_help_response_schema` generates the HelpResponse JSON Schema at runtime from the current procedure pack's step identifiers, UI map targets, and error taxonomy codes. For every overlay target in the schema's `targets` array, a conditional `if/then` sub-schema requires the `evidence` array to contain at least one item referencing that target. Each evidence item must declare a `type` from the five grounding kinds (`var`, `gate`, `rag`, `delta`, `visual`) and carry a non-empty `ref` matching the corresponding prefix family. The schema is cached keyed on configuration file modification times, avoiding redundant regeneration during long-running sessions.

Response validation and mapping. Model output passes through a narrow JSON extraction pipeline (code fence removal and delimiter-based parsing only; transformations outside this set are rejected), schema validation via `Draft202012Validator`, and allowlist checks on step identifiers and overlay targets. The mapping function validates that every overlay target cited in the response has at least one evidence reference whose prefix matches a key actually present in the help-cycle context. Targets without grounding evidence are rejected, and the rejection is recorded in the response metadata.

Safety constraints. The overlay action executor enforces three rules before dispatching to DCS: only `overlay` actions are executable (no control injection or key emulation); every target is verified against the UI target allowlist; at most one target may be highlighted concurrently. When a new highlight is requested, any prior highlight is automatically cleared.

5.3 Deterministic Fallback

The fallback mechanism operates when no model is available, model output fails JSON extraction or schema validation, or response mapping produces an empty result. It examines the currently active step, evaluates unsatisfied gating rules, identifies missing conditions, and classifies each missing evidence source as a hard blocker (gate explicitly fails) or soft blocker (telemetry source absent). This distinction prevents the tutor from incorrectly claiming a step is unsatisfied when evidence is simply unavailable. The fallback produces a text-only hint restricted to the active step; overlay highlights are suppressed unless a local rule can uniquely determine a valid target from the current telemetry state.

5.4 VLM Visual Fact Extraction Runtime

The VLM runtime operates as a pipeline triggered when the active step is vision-priority (S08, S15, S18). A capture request is dispatched to DCS via UDP; the Lua-side integration renders the three primary cockpit displays into a single composite PNG image. The Python-side buffered vision session maintains a sliding window of recent observations and selects both a pre-trigger frame (most recent frame before the trigger, within a 250 ms sync window) and the trigger frame.

The selected frames are base64-encoded and inserted alongside the fact extraction prompt into a multimodal chat-completion request. The prompt enumerates all 13 facts, describes the three-region composite-panel layout, and provides detailed decision-boundary instructions per fact. The raw model output is parsed, sanitised (unknown keys and invalid fact IDs rejected, states restricted to the three admissible values), and validated against a dynamically generated JSON Schema. Sanitised facts are merged into the runtime vision fact snapshot with sticky preservation and TTL-based expiry.

When the VLM is unavailable or the multimodal request fails, the help cycle proceeds with telemetry and knowledge only—a deliberate degradation property rather than a system failure.

5.5 Replay and Logging Infrastructure

The JSONL event store is the sole persistence mechanism for runtime events, implementing append-only writing with immediate flush after each write for crash-safe logging. The replay module provides two entry points: full simulation using a mock environment adapter that replays pre-recorded observation sequences, and log validation that reconstructs the step state machine and verifies legal state transitions against a procedure pack.

The replay evaluation infrastructure supports suite-driven regression replay: a YAML suite file specifies test cases with expected outcomes, model configuration, and optional vision frame directories. Five replay cases cover distinct scenario types (no-operation, battery-on, engine-off, FCS BIT, INS alignment), each with DCS-BIOS raw captures and per-frame sidecar files. The suite also includes 20 live-fixture regression entries that validate specific evidence-grounding and overlay-target correctness properties (Section 3.6).

Interaction metrics are derived from event logs: stall time per step, help request count,

LLM trigger count, UI click and failure counts, and highlight request and acknowledgement counts.

5.6 Live Runtime Orchestration

The live runtime orchestrates a continuous loop: DCS-BIOS UDP frames are ingested and enriched into normalised state variables; when a help trigger is received (hotkey or UDP message), the procedure engine computes a deterministic step hint, a vision capture is triggered if the active step requires it, BM25 knowledge snippets are retrieved, the full prompt is assembled and sent to the model adapter, the response is validated and mapped to overlay actions, and the complete help cycle is logged. The runtime supports both real-time DCS operation and replay of pre-recorded BIOS captures through a shared code path.

Model access is configured through environment variables with a unified naming convention; the runtime validates all required configuration at startup and redacts API keys from all log output.

6 Evaluation

This chapter presents empirical evidence organised around the three research questions defined in Chapter 1. Section 6.1 (RQ2) reports the VLM technical benchmark results on the 13-fact ontology across three backbone models. Section 6.2 (RQ1) assesses system-level technical readiness through replay evaluation and harness regression testing. Section 6.3 (RQ3) reports the formative VR pilot study with nine participants.

6.1 RQ2: VLM Technical Evaluation

The technical evaluation addresses whether a LoRA-fine-tuned VLM can reliably extract structured visual facts from F/A-18C cockpit screenshots on holdout data the model has never seen during training. All metrics are computed by the automated benchmark harness described in Section 4.6.

6.1.1 Dataset and Training Summary

The visual fact extraction task evolved across two generations of fact ontology. The first generation (v1) defined 8 visual facts and established the initial training pipeline. The second generation (v2) expanded the ontology to 13 facts, aligned with the runtime contract. Table 6.1 summarises the six annotated datasets.

Table 6.1: Summary of annotated visual fact datasets.

Dataset	Run	Facts	Tasks	Bilingual	Role
`vision_sft`	Run-001	8	180	yes	v1 train
`vision_sft_holdout_run002`	Run-002	8	50	no	v1 hold
`vision_sft_holdout_run002_newfacts`	Run-002	13	50	no	v2 hold
`vision_sft_run003`	Run-003	13	220	yes	v2 training
`vision_sft_run005_composition_rebalance`	Run-005	13	122	yes	v2 rebal
`vision_sft_holdout_run004_random`	Run-004	13	100	no	v2 hold

The v2 training recipe combines Run-003 (220 reviewed tasks) with Run-005 (122 reviewed tasks, collected to rebalance under-represented fact states), with Run-005 applied twice (Run-005×2). Bilingual export (EN + ZH) yields a combined training set of **928 rows**. The holdout sets—Run-002 newfacts (50 samples) and Run-004 random (100 samples)—are English-only with verified **zero exact overlap** with training data.

6.1.2 Qwen3.5 13-Fact V2 Results

The primary technical result is the performance of the Qwen3.5-9B LoRA adapter trained on Run-003 + Run-005×2 (928 rows, 4 epochs, learning rate 2×10^{-4}, LoRA $r = 16$, $\alpha = 16$). Table 6.2 reports results on two independent holdout sets.

Table 6.2: 13-fact v2 benchmark: Qwen3.5-9B Base vs. LoRA on two holdout sets.

	Run-002 newfacts (50)			**Run-004 random (100)**		
Metric	**Base**	**LoRA**	**Δ**	**Base**	**LoRA**	**Δ**
JSON valid	1.0	1.0	—	0.96	1.0	+0.04
Schema valid	1.0	1.0	—	0.96	1.0	+0.04
Fact accuracy	0.8677	**0.9908**	+0.1231	0.8038	**0.9931**	+0.1892
Macro F_1	0.4513	**0.6515**	+0.2002	0.4393	**0.6100**	+0.1707
Seen F_1	0.5022	**0.9648**	+0.4626	0.5659	**0.9159**	+0.3500
Exact match	0.10	**0.88**	+0.78	0.03	**0.92**	+0.89
Critical FP	11	**4**	−7	70	**8**	−62

The LoRA adapter achieves fact accuracy of 0.9908 (Run-002) and 0.9931 (Run-004), with sample exact match rates of 0.88 and 0.92 respectively. Critical false positives drop by 64% (11→4) on Run-002 and 89% (70→8) on Run-004. The base model degrades markedly on the larger Run-004 holdout (4 schema-invalid responses, 70 critical false positives concentrated on `fcsmc_intermediate_result_visible`), while the LoRA adapter restores format reliability and suppresses indiscriminate hallucination.

The residual critical false positives in the LoRA adapter are concentrated on completion-type facts: `fcsmc_final_go_result_visible` (8 on Run-004, 1 on Run-002) and `ins_ok_text_visibl` (2 on Run-002).

6.1.3 Gemma-4 and Qwen3.6 Backbone Comparison

To assess whether the training recipe generalises, the identical data recipe (Run-003 + Run-005×2, 928 rows, 13-fact ontology) was applied to two additional backbones.

Table 6.3 summarises the comparison.

Table 6.3: 13-fact v2 results across three backbones (LoRA models only).

	Run-002 newfacts (50)			Run-004 random (100)		
Metric	**Q3.5-9B**	**Q3.6-27B**	**G-4-31B**	**Q3.5-9B**	**Q3.6-27B**	**G-4-31B**
Fact accuracy	**0.9908**	0.9877	0.9492	**0.9931**	0.9892	0.9715
Seen F_1	**0.9648**	0.9479	0.8123	**0.9159**	0.9147	0.7959
Exact match	**0.88**	0.86	0.44	**0.92**	0.86	0.74
Critical FP	4	3	**0**	**8**	8	13

Qwen3.6-27B's base model starts substantially stronger than Qwen3.5-9B (fact accuracy 0.91 vs. 0.87 on Run-002) but the LoRA gain is narrower ($\Delta = +0.077$ vs. $+0.123$), consistent with the expectation that larger models have less adaptation headroom under small-data fine-tuning. The Qwen3.6 LoRA adapter achieves comparable performance to Qwen3.5 on most metrics but does not surpass the 9B line.

Gemma-4-31B achieves zero critical false positives on Run-002, reflecting a more conservative error style, but at the cost of lower recall (seen F_1 0.81 vs. 0.96 for Qwen3.5 on Run-002) and lower sample exact match (0.44 vs. 0.88). This backbone-level difference in error style—conservative (Gemma) vs. assertive (Qwen)—is relevant for safety-critical deployment decisions.

Overall, Qwen3.5-9B remains the best-performing configuration on fact accuracy, seen F_1, and sample exact match across both holdouts. The data recipe transfers across all three backbones with consistent improvements over their respective base models.

6.1.4 Error Analysis

The 13 facts can be grouped into three difficulty categories based on the Qwen LoRA adapter's performance. **Coarse panel presence** (6 facts: `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `bit_root_page_visible`, `fcsmc_page_visible`, `hsi_page_visible`): near-perfect classification, with the single exception of two false negatives on `tac_page_visible` on Run-002. **Fine-grained state indicators** (5 facts: `fcs_page_x_marks_visible`, `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible` `hsi_map_layer_visible`): perfect on four facts; residual false positives on `fcsmc_final_go_result_vi` (8 on Run-004). **Small-text recognition** (2 facts: `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`): strong on the alignment text (one false positive on Run-002); 2 false positives on `ins_ok_text_visible` on Run-002.

The base model's dominant failure mode is unidirectional hallucination on rare states: `fcsmc_intermediate_result_visible` appears in only 7 of 100 Run-004 samples, yet the base model asserts it as `seen` in 60 additional samples. The LoRA adapter eliminates this class of error almost entirely, shifting the error profile from diffuse hallucination to focused, interpretable residual errors on completion-type facts.

6.2 RQ1: System-Level Technical Readiness

Beyond the component-level VLM benchmarks, the repository contains evidence of systematic validation at the system level.

6.2.1 Replay Evaluation

The replay evaluation suite at `replay_eval/fa18c_startup_v04/` defines five test cases exercising distinct scenario types: no-operation, battery-on, battery-and-generator-off, FCS reset and BIT, and INS alignment (see Appendix 9, Table 9.4). Each case includes DCS-BIOS raw captures and, where applicable, per-frame sidecar files for vision-dependent steps. All five cases produce the expected step inference and overlay targets, confirming that the telemetry pipeline, procedure engine, and vision path function correctly end-to-end when driven by recorded data.

6.2.2 Harness Fixture Regression

The step harness (Section 3.6) is validated through 20 live-fixture regression entries in the coverage matrix, each replaying a specific telemetry snapshot through the full help cycle and asserting a specific expected outcome. The fixtures cover: correct advancement on completed conditions, safe rejection of wrong-target proposals (e.g., PB18 repaired to PB5 for BIT root sub-state, S10 left-engine guidance using left-click rather than right-click semantics), prevention of VLM evidence leakage into non-visual steps, correct handling of moving/settling controls (refuelling probe, arresting hook), consistent interaction semantics (mouse wheel for rotary controls), and correct repair of partially completed steps.

6.2.3 Experiment Export Infrastructure

The experiment export pipeline (Section 4.7) has been exercised on nine live VR trials, producing complete artifact sets (trial_summary, step_coding, help_cycles, action_timeline, quality_gate) for each. Quality gates passed for all nine trials, with configuration hashes verified consistent across the study. This demonstrates that the instrumentation pipeline is functional for human-subject data collection, though the participant/session metadata schema remains incomplete (missing questionnaire linkage fields noted in P04's quality gate warnings).

6.3 RQ3: Formative VR Pilot Study

A small formative pilot study was conducted with nine participants to gather descriptive evidence on how the tutoring system supports learners during the F/A-18C cold-start procedure compared with unaided practice. The study should be interpreted as a preliminary field trial, not a powered controlled experiment. Methodology details are reported in Section 4.7.

6.3.1 Participant and Trial Overview

Table 6.4 summarises each participant's condition, completion status, step completion, observed duration, and key interaction counts. All data are drawn from manual step coding (`step_coding.csv`) and trial summaries (`trial_summary.csv`).

Completion. All four with-tutor participants completed the full 33-step procedure. None of the five without-tutor participants completed the full procedure; they stopped after completing 19–29 steps. The most commonly missed steps in the without-tutor condition were S18 (FCS-MC BIT page, missed by 5/5 participants), S19 (FCS BIT final GO, missed by 5/5), and S27 (flap switch to AUTO, missed by 5/5).

Observed duration. With-tutor trials ranged from 317 s to 1033 s (median 387 s). Without-tutor trials ranged from 218 s to 2015 s. Because without-tutor trials ended at different procedure points when participants could not progress, these durations cannot be compared as completion times.

6.3.2 Procedural Errors

Table 6.5 reports manual error counts per participant.

Table 6.4: Participant and trial overview. Observed duration is the wall-clock interval from first telemetry event to last; it is not a completion time measure for incomplete trials.

P	Condition	Completed	Steps	Dur. (s)	Help	VLM	Ovrl.	Fallbk.
P01	without_tutor	no	23/33	2015	0	0	0	0
P02	without_tutor	no	27/33	739	0	0	0	0
P03	without_tutor	no	24/33	243	0	0	0	0
P06	without_tutor	no	29/33	269	0	0	0	0
P07	without_tutor	no	19/33	218	0	0	0	0
P04	with_tutor	yes	33/33	442	19	6	22	6
P05	with_tutor	yes	33/33	317	7	2	6	2
P08	with_tutor	yes*	33/33	332	7	5	10	4
P09	with_tutor	yes	33/33	1033	44	10	44	9

*P08: auto-summary reports incomplete; manual coding confirms 33/33 completed (see data-quality note below).

Table 6.5: Manual procedural error counts per participant (from manual step coding). OM = omission, CO = commission, OR = order error, PA = prompting/ assistance error, SV = state violation.

P	Condition	OM	CO	OR	PA	SV
P01	without_tutor	8	0	1	0	1
P02	without_tutor	0	0	0	1	0
P03	without_tutor	7	0	2	0	2
P06	without_tutor	2	0	0	2	0
P07	without_tutor	11	0	0	0	0
P04	with_tutor	0	3	0	3	1
P05	with_tutor	0	3	0	3	0
P08	with_tutor	0	0	0	0	0
P09	with_tutor	0	3	0	2	0

Omission errors. The without-tutor condition shows a clear omission pattern: four of five participants recorded 7–11 OM errors (missed steps). In contrast, the with-tutor condition recorded zero OM errors—all 33 steps were completed by all four participants.

Commission and prompting errors. The with-tutor condition shows a different error profile: CO errors (3 per participant in P04, P05, P09) and PA errors (2–3 per participant) replace the omission errors observed in the without-tutor condition. These are primarily assistance-coupled errors—cases where the tutor provided guidance that was followed but did not fully align with the procedure specification, or where the participant executed a step correctly in substance but with minor deviation from the prescribed sequence.

Error totals. Without-tutor participants averaged 7.4 manual errors; with-tutor participants averaged 4.5. However, the error categories differ qualitatively: without-tutor errors are dominated by omissions (mean OM = 5.6), while with-tutor errors are exclusively CO and PA (mean CO = 2.3, mean PA = 2.0).

6.3.3 Tutor Interaction

Table 6.6 summarises tutor interaction metrics for the four with-tutor trials.

Table 6.6: Tutor interaction summary (with-tutor condition only).

P	Help cycles	VLM calls	Overlay exec.	Overlay rej.	Fallback
P04	19	6	22	0	6
P05	7	2	6	0	2
P08	7	5	10	0	4
P09	44	10	44	0	9
Total	77	23	82	0	21

All help cycles used the model generation path (`generation_mode = model`); no cycles fell back to deterministic-only mode. VLM calls were issued for vision-priority steps (S08, S15, S18) and totalled 23 across four trials. **Overlay rejection rate was zero**: all 82 overlay actions proposed by the tutor passed evidence-grounding, allowlist, and schema validation. The 21 fallback events (21/77 = 27% of help cycles) occurred when the model's proposed overlay targets were repaired by the harness—for example, when the model proposed a target that the harness re-mapped to a more specific sub-state target (e.g., PB18 repaired to PB5 for BIT root).

6.3.4 Data-Quality Notes

1. **P08 auto-summary correction.** The automated pipeline reported P08 as incomplete (5/33 steps). Manual telemetry review confirmed all 33 steps reached required states. The auto-pipeline failed to reconstruct completion evidence for vision-priority steps from passive telemetry. Manual coding overrides the auto-summary; P08 is reported as completed with 0 manual errors.

2. **P02 questionnaire exclusion.** P02's questionnaire JSON file contains internal `participant_id = P01`, indicating mislabelling. P02 is excluded from any workload, TLX, or confidence summary. Task metrics (steps, errors) from `trial_summary.csv` and `step_coding.csv` are used.

3. **Duration interpretation.** Without-tutor trials ended at different procedure points when participants could not progress further. Observed durations are reported for completeness but do not support completion-time comparison between conditions.

4. **Non-random assignment.** Participants were assigned to conditions based on scheduling availability, not randomisation. Prior DCS and VR experience varied across participants (see Appendix 9, Table 9.5).

6.3.5 Summary of Pilot Findings

The pilot provides three pieces of formative evidence. First, all four tutor-condition participants completed the full 33-step procedure, whereas none of the five without-tutor participants did, suggesting the tutor enables procedural progression for these participants under these conditions. Second, the tutor condition eliminated omission errors in the coded data, but did not eliminate all errors: assistance-coupled CO and PA errors appeared in three of four with-tutor trials. Third, the tutoring infrastructure—telemetry grounding, overlay dispatch, evidence validation—operated without overlay rejections across 77 help cycles, supporting the technical feasibility of evidence-grounded tutoring in a live VR setting.

These findings are descriptive and formative. They do not establish that the tutor improves learning, reduces workload, or accelerates skill acquisition relative to alternative methods. The pilot's small sample ($n = 9$), non-random assignment, and variable participant experience preclude inferential conclusions. Larger controlled evaluation addressing these limitations is required.

7 Discussion

This chapter interprets the results presented in Chapter 6 and situates them within the larger trajectory of the thesis. Section 7.1 explains why the project evolved from the original VR-first proposal toward the telemetry- and VLM-heavy implementation reported here, and how the formative pilot study fits within this trajectory. Section 7.2 interprets what the benchmark results do and do not demonstrate. Section 7.3 interprets the formative VR pilot findings. Section 7.4 draws design lessons from the two-generation evolution of the visual fact ontology. Section 7.5 enumerates the known limitations of the current system and experimental evidence. Section 7.6 outlines directions for future work that follow from these limitations.

7.1 Evolution from Proposal to Implemented System

The original thesis proposal [7] specified a VR-first grounded tutoring system with gaze, hand-tracking, and voice input on the Meta Quest 3, supported by a lightweight RAG+LLM backend and evaluated through a three-condition human-subject study. The work reported in this thesis differs from that vision in two substantive ways.

First, the implementation effort shifted from VR-native interaction to telemetry-grounded tutoring on desktop DCS, because the core technical challenges—reliable telemetry processing, evidence grounding, VLM-based visual fact extraction, and harness-level output validation—required a stable development and debugging environment that desktop DCS provides. This shift produced the ports-and-adapters architecture, the procedure pack and gating engine, the deterministic fallback path, the JSONL logging and replay infrastructure, and the VLM adaptation pipeline—none of which were anticipated in the original proposal.

Second, the VLM adaptation work grew from an unplanned component into one of the thesis's main contributions. The original proposal assumed read-only simulator state flags would suffice for procedural state tracking; in practice, several critical procedure steps require visual confirmation of display-page content not exported by DCS-BIOS. Ad-

dressing this gap led to the visual fact ontology, the seven-stage data pipeline, the LoRA fine-tuning experiments, and the three-backbone benchmark comparison—forming a self-contained contribution with verified holdout evidence.

The formative VR pilot study (RQ3) represents a partial convergence back toward the original proposal's human-subject evaluation ambition. The pilot used the Quest 3 with the full tutoring runtime, produced real participant data, and demonstrated that the evidence-grounded infrastructure operates correctly in a live VR setting. However, the pilot was small ($n = 9$), non-randomised, and descriptive rather than inferential. It does not fulfil the proposal's three-condition controlled evaluation goal. The gap between the formative pilot and a fully powered controlled study remains the central unfinished thread of the thesis.

7.2 Interpretation of Technical Results

The benchmark results reported in Section 6.1 provide evidence about the performance of the VLM visual fact extraction pipeline. This section draws interpretive conclusions from those results, distinguishing carefully between what the benchmarks demonstrate and what they do not.

7.2.1 What the Benchmark Results Demonstrate

Small-data LoRA adaptation greatly improves domain-specific VLM extraction. Under the 13-fact v2 ontology, the Qwen3.5-9B LoRA adapter trained on 928 rows (Run-003 + Run-005×2) achieves a fact accuracy of 0.9908 on the Run-002 newfacts holdout and 0.9931 on the Run-004 random holdout, compared to 0.8677 and 0.8038 for the base model, respectively. The seen F_1 rises from 0.50–0.57 (base) to 0.92–0.96 (LoRA). These are large effect sizes by any standard, and they are observed on holdout sets with verified zero training overlap, ruling out memorisation as an explanation.

The data recipe transfers across model backbones. The identical training recipe (Run-003 + Run-005×2, bilingual, 13-fact ontology) applied to the Gemma-4-31B backbone produces a similar pattern of consistent improvement: fact accuracy rises from 0.82 to 0.95 on Run-002 and from 0.81 to 0.97 on Run-004. This cross-backbone transfer provides evidence that the improvement is driven by the data and ontology design rather than by idiosyncratic properties of a single model architecture.

JSON and schema validity become near-perfect after training. On the Run-

004 random holdout, the base model produces 4 schema-invalid or JSON-invalid responses out of 100; the LoRA adapter produces none. This is not a trivial property: in a runtime system that must parse model output to extract overlay targets and step recommendations, format reliability is a hard requirement. The fine-tuning process reliably teaches the model to conform to the expected output schema.

The error profile shifts from indiscriminate hallucination to targeted false positives on completion-type facts. The base model's dominant failure mode is unidirectional hallucination: it asserts `seen` for many facts that are not present, particularly on rare states such as `fcsmc_intermediate_result_visible` (60 false positives on Run-004 out of only 7 true occurrences). The LoRA adapter eliminates this class of error almost entirely, concentrating its residual errors on a specific subset of facts: `fcsmc_final_go_result_visible` (8 false positives on Run-004) and, to a lesser extent, `ins_ok_text_visible` (2 false positives on Run-002). This shift from diffuse hallucination to focused, interpretable error patterns is a qualitative improvement that is not fully captured by aggregate accuracy metrics.

The benchmark protocol itself constitutes a contribution. The use of holdout sets with verified zero overlap, per-fact confusion analysis, critical false positive tracking, and automated chart generation provides a reproducible evaluation framework that can be reused for future iterations of the ontology, for different backbone models, and for other procedures.

7.2.2 What the Benchmark Results Do NOT Demonstrate

They do not demonstrate that the tutoring system improves learning. The benchmarks measure the accuracy of visual fact extraction from cockpit screenshots. Accurate fact extraction is a necessary condition for a grounded tutoring system to function correctly, but it is not a sufficient condition for improving human learning outcomes. A learner could receive perfectly accurate visual facts and still fail to learn the procedure, or could learn effectively from a system with imperfect fact extraction. The causal chain from perception accuracy to learning outcomes has not been tested.

They do not demonstrate that visual fact extraction leads to better tutoring. The benchmarks evaluate the VLM component in isolation, not the end-to-end tutoring system. Whether the availability of visual facts improves the quality, timeliness, or acceptability of the tutor's help responses relative to a telemetry-only baseline is an open question that requires a system-level evaluation.

They measure a perception subtask, not pedagogical effectiveness. The

metrics reported — fact accuracy, seen F_1, sample exact match, critical false positives — are perception metrics. They answer the question "does the VLM see the cockpit correctly?" rather than "does the tutoring system help the learner?". Conflating the two would be a category error.

7.2.3 The `ins_go` to `ins_ok_text` Decomposition as a Design Lesson

The 8-fact v1 ontology included a single fact, `ins_go`, which asked the VLM to recognise whether the INS alignment had reached a visible GO state. This fact proved to be the dominant source of false positives in the v1 adapter: 13 of 50 Run-002 samples were incorrectly predicted as `seen`. The issue was not that the VLM failed to learn from the training data, but that the target itself was poorly specified. `ins_go` was not a visual fact in the same sense as "the FCS page is visible on the left DDI"; it was a procedural completion judgement that required integrating the presence of specific text ("OK"), the absence of other text (alignment countdown), and the procedural context (which phase of the alignment the system is in). Asking a single-frame VLM to make this judgement conflated perception with reasoning.

The 13-fact v2 ontology decomposes this compound target into lower-level, visually observable facts: `ins_grnd_alignment_text_visible` and `ins_ok_text_visible` each describe a specific piece of text on the AMPCD. The procedural interpretation — whether the combination of these facts indicates that alignment is complete — is deferred to the downstream procedure engine, which has access to telemetry, procedure phase, and timing information. This decomposition was not a cosmetic change; it was the single most important design decision in the ontology evolution, and the benchmark results bear out its effectiveness: `ins_grnd_alignment_text_visible` achieves a seen F_1 of 0.99 on Run-002, and `ins_ok_text_visible` produces only 2 false positives on Run-002 and zero on Run-004.

7.3 Interpretation of Pilot Findings

The formative VR pilot study (Section 6.3) provides descriptive evidence that must be interpreted within its proper scope.

Completion and omission errors. The clearest signal is the completion gap: 4/4 with-tutor participants completed all 33 steps, while 0/5 without-tutor participants did.

Coupled with the error coding—zero OM errors in the tutor condition versus a mean of 5.6 OM errors in the without-tutor condition—this suggests the tutor helped participants progress through the procedure by reducing missed steps. This is consistent with the tutor's design: the help system actively identifies the next required step and highlights the relevant cockpit control, making it harder for a participant to skip a step unknowingly.

Assistance-coupled errors. The tutor condition did not eliminate errors; it changed their composition. CO and PA errors appeared in three of four with-tutor trials, with a total of 9 CO and 8 PA errors across the condition. These are primarily assistance-coupled: cases where the tutor provided guidance that was followed but where the resulting action did not fully align with the procedure specification. For example, the harness occasionally repaired model-proposed overlay targets (PB18 to PB5 for BIT root sub-state), and these repairs, while correct, could produce a participant action that was technically the right control but in a slightly different context than the strict procedure definition. These errors highlight the inherent tension between flexible LLM-generated guidance and fixed procedural specifications: the tutor can guide correctly in substance while deviating in form.

Infrastructure reliability. The zero overlay rejection rate (0/82) across 77 help cycles provides evidence that the evidence-grounding, allowlist, and schema validation pipeline functions correctly in live VR operation. All 21 fallback events were harness repairs (target re-mapping), not rejections. This supports the architectural claim that evidence grounding can be enforced at the schema level without blocking legitimate tutor output.

What the pilot does not show. The pilot provides no evidence about learning (no pre/post knowledge measures), retention (no follow-up testing), workload (P02 questionnaire data unresolved; other questionnaire data not yet fully coded), or transfer. The small sample, non-random assignment, and variable participant experience mean that observed differences between conditions cannot be attributed to the tutor alone. The pilot's value is formative: it demonstrates that the instrumentation pipeline captures the intended data, that the tutor functions end-to-end in VR, and that the experimental design is executable.

7.4 Ontology Design Lessons

The two-generation evolution from 8-fact v1 to 13-fact v2 provides concrete design lessons that may inform future visual ontology construction for procedural training systems.

7.4.1 Abstraction-Level Discipline

The central lesson is that a VLM-based visual fact extractor should be asked to report *observable visual evidence*, not *procedural conclusions*. The 8-fact v1 ontology mixed these two levels of abstraction within a single fact set: facts such as `fcs_page_visible` and `bit_root_page_visible` described visually localisable display-page states, while facts such as `ins_go` and `fcs_bit_result_visible` encoded higher-level procedural judgements. The mixed-abstraction ontology produced the predictable consequence: straightforward page-visibility facts achieved strong performance, while the compound procedural facts accounted for the majority of critical false positives.

The 13-fact v2 ontology enforces a design discipline in which every fact is a locally verifiable piece of visual evidence. This discipline has three practical consequences. First, it makes the per-fact annotation task clearer for human reviewers: deciding whether "OK" text is visible on the AMPCD is a well-defined visual judgement; deciding whether "INS alignment is complete" is not. Second, it produces cleaner training signals, because the mapping from image content to fact label is more direct and less dependent on implicit procedural context. Third, it localises errors: when a fact is misclassified, the source of the error can be inspected in the image region that the fact describes, without needing to reason about the model's implicit understanding of procedure phase.

General design principle. The experience of moving from a mixed-abstraction ontology (v1) to a disciplined one (v2) yields a design principle that is explicit and transferable:

> **Design Principle:** VLM fact targets should be low-level, locally verifiable visual propositions. Higher-level procedural state judgments should be synthesised by downstream reasoning from multiple visual facts combined with temporal and telemetry context.

This principle formalises the boundary between perception and reasoning. The VLM is responsible for answering "what text, symbol, or display element is visible in this image region?"—questions that have ground-truth answers determinable from a single frame. The procedure engine is responsible for answering "what does this combination of visual

facts, telemetry variables, and temporal context imply about the current procedure state?"—questions that require multi-source integration. Enforcing this boundary at the ontology design stage, rather than discovering it through error analysis after training, is the primary design lesson of the ontology evolution.

7.4.2 Decomposition of Compound Targets

The effectiveness of this decomposition is measurable. On the Run-002 holdout, the 8-fact v1 LoRA adapter produced 15 critical false positives, of which 13 (87%) were on the compound target `ins_go`. After decomposing `ins_go` into the low-level visual propositions `ins_grnd_alignment_text_visible` and `ins_ok_text_visible`, the 13-fact v2 LoRA adapter reduced the critical false positive count from 15 to 4, with the combined residual errors on these two decomposed facts falling to 3 (1 on `ins_grnd_alignment_text_visible`, 2 on `ins_ok_text_visible`). This is not simply a change of labels; it is a measurable improvement—from 13 critical false positives to 3 on the same underlying visual content—that demonstrates the effect of applying the design principle in practice.

The `ins_go` → {`ins_ok_text_visible`, `ins_grnd_alignment_text_visible`, `hsi_map_layer_visi` decomposition is not the only example of this principle. The v1 fact `fcs_bit_result_visible` was similarly a compound target that asked the VLM to recognise a final test result rather than the visual indicators that characterise it. In v2, this is decomposed into `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, and `fcsmc_final_go_result_visi` each capturing a visually distinguishable stage in the FCS-MC test sequence. The downstream procedure engine can then interpret the sequence of these facts — together with telemetry state — to determine whether the FCS BIT is complete.

The general design principle can be stated as follows: if a human annotator cannot determine the fact label by looking at a single image without knowing the procedure step, the fact is likely too high-level and should be decomposed. This principle is operationalisable and transferable to other procedures. While demonstrated on cockpit displays, this principle applies to any VLM-based perception system that must distinguish visual presence from semantic state—for example, medical imaging (tissue visible vs. condition diagnosed), industrial inspection (gauge visible vs. value in range), and document understanding (signature field visible vs. form completed).

7.4.3 Residual Failure Modes and Their Implications

Despite the ontology redesign, completion-type facts remain the most challenging category. On the Run-004 random holdout, all 8 residual critical false positives for the Qwen LoRA adapter are on `fcsmc_final_go_result_visible`. This fact requires the model to recognise a specific numeric readout (the final GO result) and to distinguish it from visually similar intermediate numeric readouts that appear in the same screen region.

There are two plausible interpretations of this residual difficulty. One is that it is a data coverage problem: the number of training examples in which the final GO result is visible is small, and the visual distinction between intermediate and final readouts may not be sufficiently represented. The other is that it is a fundamental resolution limitation: at the image resolution used for training and inference, the difference between an intermediate numeric value and the final GO value may be genuinely below the discriminative threshold of the model. Disambiguating these two interpretations would require a controlled experiment that varies image resolution and training-set composition independently, which is beyond the scope of the current work.

7.4.4 Implications for Future Ontology Design

The experience of designing two generations of visual fact ontology for the F/A-18C cold-start suggests three guidelines for ontology construction in related procedural domains:

1. **Begin with a task analysis** that identifies which procedure states require visual confirmation and which can be covered by telemetry or rule-based inference. The ontology should only contain facts for states that genuinely require visual evidence.

2. **Enforce a single-frame verifiability test** during ontology design: every fact should be evaluable from a single cockpit image without procedural context. Facts that fail this test should be decomposed or deferred to downstream reasoning.

3. **Collect composition-rebalanced data** early in the process, particularly hard negatives that expose the boundary between visually similar but procedurally distinct states. The Run-005 composition rebalance was added specifically to address under-represented fact states, and its inclusion in the training recipe was critical to the reduction in hallucination.

7.5 Limitations

The following limitations bound the scope of conclusions that can be drawn from the present work. They are stated explicitly so that the contribution can be evaluated within its proper scope and so that future work can address them systematically.

7.5.1 Limitations of the Experimental Setup

Single procedure. All experiments, benchmarks, and dataset curation efforts focus on a single procedure: the F/A-18C cold-start. There is no evidence that the 13-fact ontology, the training recipe, or the LoRA adaptation approach generalises to other procedures (e.g., taxi, takeoff, landing, or emergency procedures in DCS, or procedural tasks in other simulators). The ontology design principles articulated in Section 7.4 are plausible but untested beyond the cold-start domain.

Single composite-panel layout. The visual fact extractor is trained and evaluated on a fixed three-display composite panel (left DDI, AMPCD, right DDI). Cockpit configurations that differ in the number, arrangement, or type of displays (e.g., the F-16C with two MFDs, or the A-10C with a different instrument layout) would require retraining with new data under a new layout description. The current models have not been tested on such configurations.

Small dataset sizes. The largest training configuration uses 928 rows (Run-003 + Run-005×2, bilingual). This is small by comparison with typical VLM fine-tuning datasets, which often use tens or hundreds of thousands of examples (e.g., LLaVA instruction tuning and domain-specific VQA fine-tuning). While the results demonstrate that 928 rows can produce meaningful improvements, they do not establish the upper bound of what is achievable with more data. Conversely, the results do not establish the minimum data requirement; it is possible that comparable performance could be achieved with fewer examples, which would be practically relevant for domains where annotated data is scarce.

Holdout sets from the same aircraft and cockpit generation. The holdout sets (Run-002 and Run-004) are independent sessions with verified zero overlap with the training data, but they are drawn from the same aircraft type, the same cockpit configuration, and the same rendering engine (DCS World). The benchmarks therefore measure *within-domain* generalisation (new sessions for the same cockpit), not *cross-domain* generalisation (new cockpit types or new simulators).

7.5.2 Limitations of the System

VR runtime operational; pilot completed but controlled study pending. The VR runtime has been exercised in a formative pilot study with nine participants (Section 6.3), confirming that the tutoring system functions end-to-end in live VR operation. However, the pilot was small ($n = 9$), non-randomised, and descriptive. In-headset overlay rendering and latency characteristics under study conditions have been observed but not systematically measured.

No controlled human-subject evaluation. The formative pilot provides descriptive evidence of feasibility but does not constitute a controlled evaluation. The pilot provides no evidence about whether the tutoring system improves learning, retention, transfer, or workload relative to baseline conditions. The causal chain from tutoring system use to improved human learning outcomes remains untested. A larger controlled study with randomised assignment, pre-registered analysis, and adequate statistical power is required to establish pedagogical value.

The benchmark measures fact extraction, not tutoring quality. The VLM benchmarks reported in Section 6.1 are perception metrics computed on isolated single-frame fact extraction. They do not measure the quality, usefulness, or timeliness of the tutor's help responses, the correctness of overlay targets, or the coherence of multi-turn help sequences. The pilot study addresses some of these gaps at a formative level, but an end-to-end evaluation of the full help cycle under controlled conditions has not been conducted.

7.5.3 Limitations of the Pilot Study

Small sample and non-random assignment. The pilot's nine participants were assigned to conditions based on scheduling availability, not randomisation. With four with-tutor and five without-tutor participants, condition sizes are small and unbalanced. No inferential statistical tests are appropriate.

Variable participant experience. Participants ranged from novice to frequent DCS users. Prior experience was not balanced across conditions, and the small sample precludes statistical control for experience effects.

Incomplete workload and questionnaire data. P02's questionnaire file was mislabelled and could not be confidently attributed; workload, TLX, and confidence data are therefore incomplete. NASA-TLX integration was not yet implemented in the export pipeline for these trials.

Without-tutor duration not comparable. Without-tutor trials ended at different procedure points (19–29 steps completed), so observed durations do not measure completion time and cannot be compared with with-tutor durations.

Passive visual verification limitation. For without-tutor trials, step completion was inferred from passive telemetry gates and action timeline review, without active VLM visual confirmation. Steps requiring visual evidence (S08, S18) may be misclassified as completed or not completed based on incomplete telemetry evidence alone.

7.5.4 Limitations of the VLM Adaptation Results

Residual critical false positives on completion-type facts. As discussed in Section 7.4, the Qwen LoRA adapter retains critical false positives on `fcsmc_final_go_result_visible` (1 on Run-002, 8 on Run-004). In a runtime deployment, each such false positive carries the risk that the tutoring system incorrectly concludes that the FCS-MC test is complete and advances the procedure prematurely. Mitigation strategies — such as the sticky-fact TTL mechanism and the requirement for multiple confirming observations — exist but have not been evaluated end-to-end.

Limited model diversity. Two backbone models were evaluated (Qwen3.5-9B and Gemma-4-31B), both with LoRA fine-tuning using the Unsloth + PEFT + TRL stack. The results do not generalise to other model architectures (e.g., LLaVA-style models, InternVL), other fine-tuning methods (e.g., full fine-tuning, QLoRA with different rank configurations), or other training frameworks.

No ablation of training components. The training recipe combines several design choices — bilingual SFT, Run-005×2 oversampling, composition rebalancing, LoRA rank and alpha settings — without isolating the contribution of each. It is therefore unclear which components are necessary and which are incidental. A systematic ablation study would clarify the design space but has not been conducted.

7.5.5 Scope Summary

These limitations bound the claims of this thesis. Within these bounds, the thesis demonstrates: (1) a working dual-platform tutoring runtime with automated tests and harness-level evidence validation; (2) a reproducible VLM adaptation pipeline achieving 0.99 fact accuracy on holdout data with cross-backbone evidence; (3) a pack-driven architecture supporting extension to new procedures and simulators; and (4) formative VR pilot evidence ($n = 9$) that the tutor was associated with full procedural comple-

tion and fewer coded omission errors, while assistance-coupled errors persisted. These contributions do not depend on claims that exceed the stated bounds.

7.6 Future Work

The limitations enumerated above define a concrete research agenda. This section outlines the principal directions for future work, ordered from the most immediate technical extensions to the longer-term thesis objectives.

7.6.1 Controlled Human-Subject Evaluation

The formative pilot study (Section 6.3) demonstrated feasibility and instrumentation readiness but did not provide controlled evidence about learning, workload, or retention. The next research stage is a larger controlled between-subjects evaluation comparing the grounded tutor against an appropriate baseline condition (e.g., without-tutor or static checklist). This requires completing the participant/session metadata schema, NASA-TLX integration, quiz linkage, and anonymisation pipeline; recruiting an adequate sample with randomised assignment; and pre-registering the analysis plan. Given the pilot's observed effect on completion rates, a two-condition design (with-tutor vs. without-tutor) with $n \geq 15$ per condition would provide a more feasible starting point than the original three-condition proposal design.

7.6.2 Extending the Visual Fact Ontology to Other Procedures

The current 13-fact ontology is specific to the F/A-18C cold-start. Extending the approach to other procedures — within DCS (taxi, takeoff, landing, aerial refuelling, emergency procedures) or in other simulators — would test the generality of the ontology design principles proposed in Section 7.4. Each new procedure would require a task analysis to identify vision-priority steps, a new fact ontology designed under the single-frame verifiability constraint, capture and annotation of new training data, and independent holdout evaluation. Whether the data recipe (bilingual SFT, composition rebalancing, moderate oversampling) transfers to procedurally different domains is an open empirical question.

7.6.3 Multi-Frame Temporal Reasoning

The current visual fact extraction operates on single composite-panel images. Several cockpit states of interest involve temporal dynamics that cannot be disambiguated from a single frame: countdown values on the INS alignment page, animation sequences on the FCS-MC test display, and transient warning indicators. Extending the VLM input to short frame sequences (e.g., 3–5 consecutive frames at 1–2-second intervals) could enable the model to capture these temporal cues directly. This extension would require changes to the data capture pipeline (recording frame sequences rather than isolated screenshots), the training format (video or multi-image inputs), and the benchmark protocol (temporal fact definitions). Whether current small-scale VLMs can effectively exploit temporal information for cockpit state recognition is uncertain. Whether multi-frame VLM input is feasible at the 9B-parameter scale under LoRA training has not been tested.

7.6.4 Online Adaptation from In-Situ Corrections

The current VLM adaptation pipeline is an offline process: data is captured, pre-labeled, reviewed, and used to train a LoRA adapter in a batch cycle. In a deployed tutoring system, a learner or instructor may correct the system's visual fact predictions in real time (e.g., "the OK text is not actually visible yet"). Collecting these in-situ corrections and using them for online or incremental model adaptation — without requiring a full re-labeling and re-training cycle — could enable the system to improve its extraction accuracy over the course of a single training session or across multiple learners. This direction requires addressing challenges in label reliability (learner corrections may themselves be erroneous), catastrophic forgetting (online updates should not degrade performance on previously correct facts), and evaluation (online adaptation must be benchmarked against a fixed holdout set that remains untouched by the adaptation process). Whether the current Unsloth + PEFT + TRL stack supports incremental fine-tuning without full retraining has not been evaluated.

7.6.5 System-Level End-to-End Evaluation

Beyond the component-level VLM benchmarks reported in this thesis, a system-level evaluation of the full help cycle is needed. This would measure, under replay or live conditions: the end-to-end latency from help trigger to overlay rendering; the correctness of overlay targets on vision-priority steps; the rate of deterministic fallback (when model

output fails validation); and the perceived usefulness of help responses as judged by domain experts reviewing replay logs. Such an evaluation would bridge the gap between the component-level benchmarks (reported here) and the pilot study (Section 6.3), providing evidence about whether the system's components function correctly when integrated.

7.6.6 Summary of Future Directions

Table 7.1 summarises the principal future directions and their current status.

Table 7.1: Summary of future work directions.

Direction	**Current status**	**Key challenge**
Controlled human-subject evaluation	Formative pilot completed ($n = 9$); controlled study pending	Recruitment, randomisation, statistical power
Ontology extension to other procedures	13-fact v2 validated for cold-start only	Task analysis, annotation effort, domain generality
Multi-frame temporal reasoning	Single-frame pipeline operational	VLM capacity, training data volume, benchmark design
Online adaptation from corrections	Not implemented	Label reliability, catastrophic forgetting, evaluation
System-level end-to-end evaluation	Not conducted	Defining meaningful system-level metrics

The work presented in this thesis provides the technical foundation for each of these directions. The architecture, data pipeline, training infrastructure, and benchmark protocol are designed to be extensible, and the lessons learned from the two-generation ontology evolution and the cross-backbone comparison provide evidence-based guidance for the next stages of the project.

8 Conclusion

This thesis set out to design, implement, and technically validate a telemetry-grounded tutoring runtime for procedural learning in a high-fidelity flight simulator, and to gather formative evidence about its use in immersive VR training. The three contributions delivered are: an evidence-grounded tutoring architecture, a visual fact ontology and small-data VLM adaptation pipeline with cross-backbone benchmark evidence, and a VR evaluation framework exercised in a formative pilot study ($n = 9$). This chapter summarises the completed work against the three research questions, restates the contributions, and outlines directions for future work.

8.1 Summary of Completed Work

The thesis has produced three contributions, corresponding to the three research questions defined in Chapter 1.

C1 — An evidence-grounded procedural tutoring architecture. The tutoring runtime enforces that every overlay target and step recommendation carries a traceable evidence reference (telemetry variable, gating rule, RAG snippet, or visual fact), enforced at the schema level through runtime-generated JSON Schema with `if/then` rules—not merely logged for post-hoc audit. The architecture follows a ports-and-adapters pattern with a deterministic fallback path, a step harness that validates and repairs model outputs before they reach the simulator, and a procedure pack format that expresses all procedure logic as declarative YAML configuration. The architecture is validated through automated tests, a replay evaluation suite (5 cases), and 20 live-fixture regression entries covering correct advancement, safe rejection, and interaction semantics. In the VR pilot, the architecture processed 77 help cycles with zero overlay rejections.

C2 — A visual fact ontology and small-data VLM adaptation pipeline. The 13-fact v2 ontology decomposes cockpit display-page state into locally verifiable propositions. The ontology evolved from an 8-fact v1 baseline that mixed abstraction levels, producing systematic false positives on completion-type facts. The v2 decomposition—

breaking `ins_go` into `ins_grnd_alignment_text_visible` and `ins_ok_text_visible`, and decomposing the FCS-MC sequence into four sub-states—reduced critical false positives by 64% (13→4) on Run-002 and 89% (70→8) on Run-004. A Qwen3.5-9B LoRA adapter trained on 928 bilingual rows achieves 0.99 fact accuracy on both holdout sets. The identical data recipe transfers across three backbones (Qwen3.5-9B, Qwen3.6-27B, Gemma-4-31B) with consistent improvements over their respective base models. The ontology evolution yields a transferable design principle: VLMs should extract observable visual evidence, while procedural interpretation belongs in downstream reasoning.

C3 — A VR evaluation framework supported by a formative pilot. The thesis delivers study-ready instrumentation for evaluating grounded procedural tutoring in immersive simulation, and a formative VR pilot study ($n = 9$) that exercised this instrumentation. All four tutor-condition participants completed the full 33-step procedure; none of the five without-tutor participants did. With-tutor errors were predominantly assistance-coupled (CO/PA), while without-tutor errors were predominantly omissions. The pilot provides descriptive evidence of feasibility and instrumentation readiness, not inferential validation of learning effects. Larger controlled evaluation remains future work.

What this thesis demonstrates. The work establishes that a telemetry-grounded, evidence-constrained tutoring runtime with structured visual fact extraction is technically feasible and can be validated through systematic benchmarks. It demonstrates that 928 rows of human-reviewed data suffice to adapt a general-purpose VLM to a domain-specific extraction task with near-ceiling accuracy on independent holdout sets. It shows that visual fact ontology design—specifically, the decomposition of compound procedural targets into locally verifiable atoms—has a measurable, quantitative impact on model reliability. The cross-backbone results confirm that the data recipe, not a specific architecture, drives the gains. The formative VR pilot provides initial descriptive evidence that full procedural completion was observed in all tutor-condition trials while omission errors were less frequent, and supports the readiness of the instrumentation pipeline for live VR data collection.

Scope and boundaries. The VLM benchmarks measure fact extraction accuracy, not pedagogical effectiveness. The VR pilot provides formative descriptive evidence ($n = 9$, non-randomised, unbalanced conditions) and does not establish learning gains, workload reduction, or retention improvement. Without-tutor observed durations are not compar-

able with with-tutor durations because trials were stopped at different procedure points. Questionnaire and workload data remain incomplete (P02 questionnaire mislabelled). Within these boundaries, the thesis delivers a technically validated tutoring runtime, a reproducible VLM adaptation pipeline with benchmark evidence, and formative VR pilot results that demonstrate feasibility and instrumentation readiness.

8.2 Future Work

The contributions of this thesis open several directions for future work, each of which builds directly on the completed contributions.

Controlled human-subject evaluation. The formative VR pilot study (Section 6.3) demonstrated feasibility and instrumentation readiness. The next step is a larger controlled between-subjects evaluation comparing the grounded tutor against an appropriate baseline, with randomised assignment, pre-registered analysis, and adequate sample size. Key infrastructure upgrades required include completion of the participant/session metadata schema, NASA-TLX integration, quiz linkage, and anonymisation pipeline. Given the pilot's observed completion-rate pattern, a two-condition design (with-tutor vs. without-tutor) with $n \geq 15$ per condition provides a feasible starting point.

Ontology extension to other procedures. The 13-fact v2 ontology is specific to the F/A-18C cold-start task and its three-display composite panel layout. Extending the ontology to other procedures within DCS—such as navigation, weapon employment, or emergency checklists—would test the generality of the visual fact decomposition strategy and require new dataset capture and annotation cycles. The procedure pack format is designed to accommodate such extensions: a new pack directory with its own `vision_facts.yaml`, `vision_layout.yaml`, `step_registry.yaml`, and `telemetry_map.yaml` would define a new training scenario without changes to the pipeline tools.

Multi-frame temporal reasoning. The current VLM pipeline operates on single frame pairs (pre-trigger and trigger frames) and makes independent predictions per observation window. Incorporating temporal context across multiple frames could improve accuracy on facts that involve transient states or state transitions, such as the FCS-MC test sequence (`fcsmc_in_test_visible` → `fcsmc_final_go_result_visible`). Pos-

sible approaches include frame stacking as multi-image input to the VLM, recurrent aggregation of fact predictions over a sliding window, or a lightweight temporal model that consumes per-frame fact vectors and produces temporally consistent sequences.

Online adaptation. The current LoRA adapter is trained offline on a fixed dataset and deployed as a static model. An online adaptation loop, in which the system collects new labelled frames during live tutoring sessions (with tutor or expert verification) and periodically updates the adapter, could progressively improve accuracy on edge cases and adapt to changes in cockpit display configurations across DCS versions or aircraft variants. This would require lightweight fine-tuning infrastructure that can operate concurrently with the tutoring runtime and a quality-control mechanism for newly acquired labels.

Deterministic procedure engine enhancements. The current gating rule engine uses a v1 DSL with four operators (`var_gte`, `arg_in_range`, `flag_true`, `time_since`). Extending the DSL with additional operators (e.g., logical combinations, sequence counters, multi-variable conditions) and supporting user-defined gating functions would increase the expressiveness of procedure packs and reduce reliance on manual-step classification for edge cases.

The work completed in this thesis establishes that a telemetry-grounded, evidence-constrained tutoring runtime with structured visual fact extraction is technically feasible and can be validated through automated benchmarks. The next phase of research must determine whether this capability translates into improved human learning in immersive simulation environments.

References

[1] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.

[2] Eagle Dynamics. DCS World, 2013. Digital Combat Simulator. Available at: https://www.digitalcombatsimulator.com/.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[4] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods, 2022. URL https://github.com/huggingface/peft.

[5] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011. doi: 10.1080/00461520.2011.611369.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[7] Yu Zhang. Explain reality: Grounded procedural tutoring with foundation models and visual fact extraction in immersive simulation — MSc thesis proposal. TU Clausthal, Institute for Software and Systems Engineering (ISSE). Unpublished manuscript., 2025.

9 Appendix

9.1 Full LoRA Hyperparameter Configuration

Table 9.1 lists the complete hyperparameter configuration for all three backbone models under the matched Run-003 + Run-005×2 recipe. Values are drawn from available training reports and benchmark artifacts.

Table 9.1: Complete LoRA fine-tuning hyperparameters across three backbones.

Parameter	**Qwen3.5-9B**	**Qwen3.6-27B**	**Gemma-4-31B**
`max_seq_length`	4096	4096	4096
`num_train_epochs`	4.0	4.0	4.0
`learning_rate`	2×10^{-4}	2×10^{-4}	2×10^{-4}
`per_device_train_batch_size`	1	1	1
`gradient_accumulation_steps`	4	4	4
`lora_r`	16	16	16
`lora_alpha`	16	16	16
`lora_dropout`	0.0	0.0	0.0
`seed`	3407	3407	3407
`load_in_4bit`	True	True	True
`warmup_ratio`	0.1	0.1	0.1
`weight_decay`	0.01	0.01	0.01
`train_rows`	928	928	928
`eval_rows`	0	0	0
Backbone-specific			
Vision LoRA	enabled	enabled	disabled
LoRA target modules	vision+lang	vision+lang	all-linear
Chat template	Qwen default	Qwen default	gemma-4
GPU memory util.	0.6	0.6	0.95
Training outcomes			
Train runtime (s)	6419	12619	8050
Train loss (final)	0.2156	0.1068	0.0474

Note: training loss values are not directly comparable across backbones due to differ-

ences in tokenizer behaviour, chat-template structure, target-sequence distribution, and model parameterisation.

9.2 8-Fact V1 Baseline Benchmark

The earliest complete benchmark uses an 8-fact LoRA adapter trained on Run-001 (180 tasks) and evaluated on the Run-002 holdout (50 samples). This line predates the 13-fact ontology and is included for historical reference.

Table 9.2: 8-fact v1 benchmark: Qwen3.5-9B Base vs. LoRA-v1 on Run-002 holdout (50 samples, EN).

Metric	**Base**	**LoRA-v1**	**Δ**
JSON valid rate	1.0	1.0	—
Schema valid rate	1.0	1.0	—
Fact accuracy	0.7600	0.9150	+0.1550
Macro F_1	0.4241	0.5847	+0.1605
Seen F_1	0.4714	0.8380	+0.3666
Sample exact match	0.06	0.36	+0.30
Critical false positives	13	15	+2

The LoRA adapter improves fact-level metrics substantially but increases critical false positives from 13 to 15, confirming the 8-fact v1 ontology's structural weakness: compound targets such as `ins_go` were not visually verifiable from single frames and produced systematic false-positive errors on completion-type facts.

9.3 Dataset Fact Distributions

Table 9.3 reports per-fact `seen` counts for the two main training sources (Run-003, Run-005) and the two 13-fact holdout sets (Run-002 newfacts, Run-004 random). Counts are drawn from the corresponding `stats.json` files.

The Run-005 composition rebalance successfully improved coverage of underrepresented facts: `supt_page_visible` from 20 to 54 combined seen, `fcsmc_page_visible` from 55 to 111, and `ins_grnd_alignment_text_visible` from 58 to 130.

Table 9.3: Per-fact `seen` counts across training and holdout datasets (13-fact v2 ontology).

Fact ID	Run-003 (220)	Run-005 (122)	Run-002 new (50)	Run-
tac_page_visible	46	24	23	
supt_page_visible	20	34	5	
fcs_page_visible	30	40	13	
fcs_page_x_marks_visible	16	18	1	
bit_root_page_visible	18	43	9	
fcsmc_page_visible	55	56	9	
fcsmc_intermediate_result_visible	18	15	1	
fcsmc_in_test_visible	18	19	6	
fcsmc_final_go_result_visible	18	22	2	
hsi_page_visible	106	109	49	
hsi_map_layer_visible	79	37	27	
ins_grnd_alignment_text_visible	58	72	42	
ins_ok_text_visible	12	14	4	

9.4 CLI Command Reference

The `simtutor` package provides the following CLI commands:

`run`. Executes a simulation using a mock environment adapter with a pre-recorded scenario file and writes the event log.

`replay`. Validates an existing event log against a procedure pack, checking step ordering and state-machine consistency.

`score`. Scores an event log using the scoring engine, producing omission counts, state-violation counts, and a total error score.

`record-vlm`. Runs VLM fact extraction on a set of vision observations and records predictions for benchmarking.

`replay-bios`. Replays a raw DCS-BIOS capture through the telemetry pipeline and optionally invokes the help generation loop.

`validate`. Validates JSONL files against a named schema from the schema registry.

All commands accept model configuration overrides (`-model-provider`, `-model-name`, `-model-base-url`, `-model-timeout-s`) and a language switch (`-lang`).

9.5 Model Provider Configuration

Model access is configured through environment variables: `SIMTUTOR_MODEL_PROVIDER` (`openai_compat` | `ollama` | `stub`), `SIMTUTOR_MODEL_BASE_URL`, `SIMTUTOR_MODEL_NAME`, `SIMTUTOR_MODEL_TIMEOUT_S`, `SIMTUTOR_MODEL_API_KEY` (required for `openai_compat`), `SIMTUTOR_MODEL_ENABLE_MULTIMODAL` (toggles base64 image encoding in requests), and `SIMTUTOR_LANG` (`zh` | `en`). The live runtime validates all required configuration at startup and redacts API keys from all log output.

9.6 Replay Evaluation Suite

Table 9.4 lists the five replay evaluation cases in the `fa18c_startup_v04` suite.

Table 9.4: Replay evaluation cases.

Case ID	Expected step	Vision	Expected target
`noop_2min`	S01	unavailable	`battery_switch`
`batteryon_2min`	S02	unavailable	`fire_test_switch`
`Batteryon_enginGenoff_2min`	S01	unavailable	`generator_left_switch`
`fcs_reset_fcs_bit_2min`	S18	available	`fcs_bit_switch`
`ins_2min`	S12	available	`ins_mode_knob`

Cases with `vision_status = available` include per-frame sidecar files (`frames.jsonl`) in the corresponding DCS Saved Games directory, enabling replay of vision-dependent procedure steps without a live DCS instance.

9.7 VR Pilot Study — Supplementary Data

Table 9.5 lists participant experience profiles.

Table 9.6 lists the specific steps not completed by each without-tutor participant.

Table 9.7 reports the auto-coded error counts (before manual review) alongside the manual error counts, illustrating the magnitude of auto-to-manual correction in this pilot.

Table 9.5: Pilot participant experience profiles (self-reported).

P	Condition	DCS exp.	Flight sim.	VR exp.	Quest 3	Group
P01	without_tutor	frequent	frequent	frequent	frequent	intermediate
P02	without_tutor	intermediate	intermediate	frequent	frequent	—
P03	without_tutor	novice	novice	intermediate	frequent	beginner
P04	with_tutor	intermediate	intermediate	frequent	frequent	intermediate
P05	with_tutor	intermediate	intermediate	frequent	frequent	—
P06	without_tutor	novice	novice	intermediate	frequent	beginner
P07	without_tutor	novice	novice	intermediate	frequent	beginner
P08	with_tutor	novice	novice	frequent	frequent	—
P09	with_tutor	novice	novice	intermediate	intermediate	beginner

— = not recorded.

Table 9.6: Steps not completed in the without-tutor condition (manual coding).

P	Steps not completed
P01	S09, S12, S18, S19, S20, S22, S24, S27, S29, S31
P02	S02, S14, S18, S19, S27, S30
P03	S07, S09, S16, S18, S19, S26, S27, S29, S32
P06	S07, S18, S19, S27
P07	S02, S09, S12, S13, S16, S17, S18, S19, S22, S26, S27, S28, S29, S32

S18 (FCS-MC BIT page on right DDI) and S19 (FCS BIT final GO result) are missed by all five without-tutor participants. S27 (flap switch to AUTO) is missed by all five. These steps are vision-priority and require either VLM confirmation or instructor knowledge of the correct display-page sequence.

Table 9.7: Auto-coded vs. manual error counts per participant.

P	Condition	Auto OM	Auto CO	Manual OM	Manual CO	Total manual
P01	without_tutor	4	3	8	0	10
P02	without_tutor	0	3	0	0	1
P03	without_tutor	0	4	7	0	11
P06	without_tutor	0	3	2	0	4
P07	without_tutor	1	9	11	0	11
P04	with_tutor	0	4	0	3	7
P05	with_tutor	0	3	0	3	6
P08	with_tutor	0	3	0	0	0
P09	with_tutor	0	3	0	3	5

Auto-coded errors are from the automated pipeline (Auto_Error_* columns in `step_coding.csv`). Manual errors are from the human-reviewed columns (Error_*). P08's auto errors were all overridden to zero by manual review (see data-quality note in Section 6.3).